arXiv:2401.00909v2 [cs.CV] 29 Mar 2024

Taming Mode Collapse in Score Distillation for Text-to-3D Generation

Peihao Wang[1,*], Dejia Xu[1], Zhiwen Fan[1], Dilin Wang[2], Sreyas Mohan[2], Forrest Iandola[2], Rakesh Ranjan[2], Yilei Li[2], Qiang Liu[1], Zhangyang Wang[1], Vikas Chandra[2]
[1]The University of Texas at Austin, [2]Meta Reality Labs
{peihaowang, dejia, zhiwenfan, atlaswang}@utexas.edu, lqiang@cs.utexas.edu
{wdilin, sreyasmohan, fni, rakeshr, yileil, vchandra}@meta.com
vita-group.github.io/3D-Mode-Collapse/

Abstract

Despite the remarkable performance of score distillation in text-to-3D generation, such techniques notoriously suffer from view inconsistency issues, also known as "Janus" artifact, where the generated objects fake each view with multiple front faces. Although empirically effective methods have approached this problem via score debiasing or prompt engineering, a more rigorous perspective to explain and tackle this problem remains elusive. In this paper, we reveal that the existing score distillation-based text-to-3D generation frameworks degenerate to maximal likelihood seeking on each view independently and thus suffer from the mode collapse problem, manifesting as the Janus artifact in practice. To tame mode collapse, we improve score distillation by re-establishing the entropy term in the corresponding variational objective, which is applied to the distribution of rendered images. Maximizing the entropy encourages diversity among different views in generated 3D assets, thereby mitigating the Janus problem. Based on this new objective, we derive a new update rule for 3D score distillation, dubbed Entropic Score Distillation (ESD). We theoretically reveal that ESD can be simplified and implemented by just adopting the classifier-free guidance trick upon variational score distillation. Although embarrassingly straightforward, our extensive experiments demonstrate that ESD can be an effective treatment for Janus artifacts in score distillation.

1. Introduction

Recent advancements in text-to-3D technology have attracted considerable attention, particularly for its pivotal role in automating high-quality 3D content. This is especially crucial in fields such as virtual reality and gaming, where 3D content forms the bedrock. While numerous techniques are available, the prevailing text-to-3D approach is based on score distillation [31], popularized by DreamFusion and its follow-up works [4, 19, 26, 50, 54, 56].

Score distillation leverages a pre-trained 2D diffusion model to sample over the 3D parameter space (*i.e.* Neural Radiance Fields (NeRF) [27]) such that views rendered from a random angle satisfy the statistics of the image distribution. This algorithm is implemented by backpropagating the estimated score of each view via the chain rule. Despite the notable progress achieved with score distillation-based approaches, it is widely observed that 3D content generated using score distillation suffers from the *Janus* problem [12], referring to the artifacts that generated 3D objects contain multiple canonical views (see Fig. 1).

To understand this drawback of score distillation, we draw the theoretical connection between the Janus problem and *mode collapse*, a statistical term describing a distribution concentrating on the high-density area while losing information about the probability tail. We first uncover that the optimization of existing score distillation-based text-to-3D generation degenerates to a maximum likelihood objective, making it susceptible to model collapse. As pre-trained diffusion models are biased to frequently encountered views [12][1], this oversight leads all views opt to convergence toward the point with the highest likelihood, manifesting as the Janus artifact in practical applications. The main limitation of current methods is that their distillation objectives solely maximize the likelihood of each view independently, without considering the diversity between different views.

To address the aforementioned issue, we propose a principled approach *Entropic Score Distillation* (ESD), which regularizes the score distillation process by entropy maximization of the rendered image distribution, thereby enhancing the diversity of views in generated 3D assets and alleviating the Janus problem. Our derived ESD update admits a simple form as a weighted combination of scores for pre-trained image distribution and rendered image distribution. Com-

[*]Work done during an internship with Meta.

[1]For example, it is common that a frontal view of a cat is more likely to be sampled from latent diffusion models than the back view.

Figure 1. **A Preview of Qualitative Results.** We present the front and back views of objects synthesized by VSD (ProlificDreamer) on the right two columns, and four views of our generated results on the left. VSD suffers from "Janus" problem, where both front and back views contain a frontal face of the targeted object, while our method effectively mitigates this artifact. Please refer to more results in Appendix D.

pared with Score Distillation Sampling (SDS) [31], our ESD involves the score of the rendered image distribution, serving to maximize the entropy of the rendered image distribution. Unlike Variational Score Distillation (VSD) [56], the learned score function of the rendered image distribution does not depend on the camera pose. This subtle difference has a more profound impact, as we show the score function of rendered images modeled by VSD corresponds to an objective with fixed entropy, thereby having no influence on view variety. In contrast, ESD optimizes for a Kullback-Leibler divergence with a non-constant entropy term parameterized by the 3D model, leading to an effect that encourages diversity among different views.

In practice, we find it challenging to optimize the score of the rendered image distribution without conditioning on the camera pose. To facilitate training, we discover that the gradient from the entropy can be decomposed into a combination of scores: one depends on the camera pose, and the other independent of it, with a coefficient interacting between these two terms. Through this theoretical establishment, we are able to adopt a handy implementation of ESD by Classifier Free Guidance (CFG) trick [10] where conditional and unconditional scores are trained alternatively and mixed during inference.

Through extensive experiments with our proposed ESD, we demonstrate its efficacy in alleviating the Janus problem and its significant advantages in improving 3D generation quality when compared to the baseline methods [31, 56] and other remedy techniques [2, 12]. As a side contribution, we also borrow two inception scores [36] to evaluate text-to-3D results and numerically probe model collapse in score distillation. We show these two metrics can effectively characterize the quality and diversity of views, highly matching our qualitative observations.

2. Background

2.1. Diffusion Models

Diffusion models, as demonstrated by various works [11, 44, 46, 48], have shown to be highly effective in text-to-image generation. Technically, a diffusion model learns to gradually transform a normal distribution $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$ to the target distribution $p_{data}(\boldsymbol{x}|\boldsymbol{y})$ where $\boldsymbol{y}$ denotes the text prompt embeddings. The sampling trajectory is determined by a forward process with the conditional probability $p_t(\boldsymbol{x}_t|\boldsymbol{x}_0) = \mathcal{N}(\boldsymbol{x}_t|\alpha_t\boldsymbol{x}_0, \sigma_t^2\boldsymbol{I})$, where $\boldsymbol{x}_t \in \mathbb{R}^D$ represents the sample at time $t \in [0, T]$, and $\alpha_t, \sigma_t > 0$ are time-dependent diffusion coefficients. Consequently, the distribution at time t can be formulated as $p_t(\boldsymbol{x}_t|\boldsymbol{y}) = \int p_{data}(\boldsymbol{x}_0|\boldsymbol{y})\mathcal{N}(\boldsymbol{x}_t|\alpha_t\boldsymbol{x}_0, \sigma_t^2\boldsymbol{I})d\boldsymbol{x}_0$. Diffusion models generate samples through a reverse process starting from Gaussian noises, which can be described by the ODE:

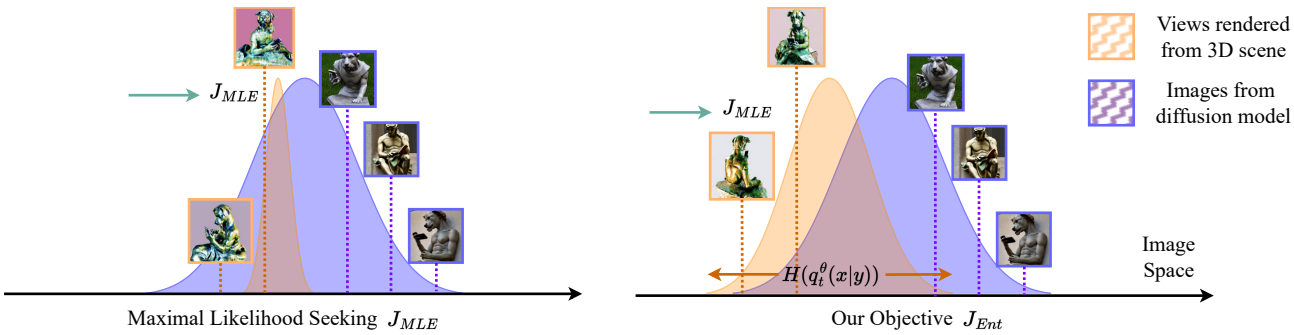

Figure 2. **Illustration of the effect of entropy regularization.** Learned image distributions often exhibit a higher probability mass for objects' frontal faces. Pure maximal likelihood seeking is opt to mode collapse (Sec. 3). Adding entropy regularization can expand the support of fitted distribution $q_t^{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{y})$ with mode-covering behavior (Sec. 4).

$\mathrm{d}\boldsymbol{x}_t/\mathrm{d}t = -\nabla_{\boldsymbol{x}} \log p_t(\boldsymbol{x}_t)$ with the boundary condition $\boldsymbol{x}_T \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ [22, 45, 48]. Such a process requires the computation of *score function* $\nabla_{\boldsymbol{x}} \log p_t(\boldsymbol{x}_t)$ which is often obtained by fitting a time-conditioned noise estimator $\boldsymbol{\epsilon}_{\boldsymbol{\phi}} : \mathbb{R}^D \to \mathbb{R}^D$ using score matching loss [15, 47, 52].

2.2. Text-to-3D Score Distillation

Score distillation based 3D asset generation requires representing 3D scenes as learnable parameters $\boldsymbol{\theta} \in \mathbb{R}^N$ equipped with a differentiable renderer $g(\boldsymbol{\theta}, \boldsymbol{c}) : \mathbb{R}^N \to \mathbb{R}^D$ that projects 3D scene $\boldsymbol{\theta}$ into images with respect to the camera pose $\boldsymbol{c}$. Here N, D are the dimensions of the 3D parameter space and rendered images, respectively. Neural radiance fields (NeRF) [27] are often employed as the underlying 3D representation for its capability of modeling complex scenes.

Recent works [4, 14, 19, 26, 31, 50, 54–56] demonstrate the feasibility of using a pretrained 2D diffusion model to guide 3D object creation. Below, we elaborate on two score distillation schemes, adopted therein: *Score Distillation Sampling* (SDS) [31] and *Variational Score Distillation* (VSD) [56].

Score Distillation Sampling (SDS). SDS updates the 3D parameter $\boldsymbol{\theta}$ as follows [2]:

$$\nabla_{\boldsymbol{\theta}} J_{SDS}(\boldsymbol{\theta}) = -\mathbb{E}\left[\omega(t)\frac{\partial g(\boldsymbol{\theta}, \boldsymbol{c})}{\partial \boldsymbol{\theta}}\left(\sigma_t \nabla \log p_t(\boldsymbol{x}_t|\boldsymbol{y}) - \boldsymbol{\epsilon}\right)\right], \tag{1}$$

where the expectation is taken over timestep $t \sim \mathcal{U}[0, T]$, Gaussian noises $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$, and camera pose $\boldsymbol{c} \sim p_c(\boldsymbol{c})$. Here is $\nabla \log p$ is a pre-trained diffusion model $\boldsymbol{\epsilon}_{\boldsymbol{\phi}}(\boldsymbol{x}, t, \boldsymbol{y})$ and $\boldsymbol{x}_t$ is a noisy version of the rendering given by camera pose $\boldsymbol{c}$. $\boldsymbol{x}_t = \alpha_t g(\boldsymbol{\theta}, \boldsymbol{c}) + \sigma_t \boldsymbol{\epsilon}$. Updating $\boldsymbol{\theta}$ as in Eq. (1) has been shown to minimize the evidence lower bound (ELBO) for the rendered images, see Wang et al. [54], Xu et al. [59].

[2]Without special specification, expectations are taken over all relevant random variables and Jacobian matrices are transposed by default.

Variational Score Distillation (VSD). VSD [56] is introduced in ProlificDreamer, VSD improves upon SDS by deriving the following Wasserstein gradient flow [51]:

$$\nabla_{\boldsymbol{\theta}} J_{VSD}(\boldsymbol{\theta}) = -\mathbb{E}\left[\omega(t)\frac{\partial g(\boldsymbol{\theta}, \boldsymbol{c})}{\partial \boldsymbol{\theta}}\left(\sigma_t \nabla \log p_t(\boldsymbol{x}_t|\boldsymbol{y}) - \sigma_t \nabla \log q_t(\boldsymbol{x}_t|\boldsymbol{c})\right)\right]. \tag{2}$$

Similarly, $\boldsymbol{x}_t = \alpha_t g(\boldsymbol{\theta}, \boldsymbol{c}) + \sigma_t \boldsymbol{\epsilon}$ is the noisy observation of the rendered image. In contrast to SDS, VSD introduces a new score function of the noisy rendered images conditioned on the camera pose $\mathbf{c}$. To obtain this score, Wang et al. [56] fine-tunes a diffusion model using images rendered from the 3D scene as follows:

$$\min_{\boldsymbol{\psi}} \mathbb{E}\left[\omega(t)\|\boldsymbol{\epsilon}_{\boldsymbol{\psi}}(\alpha_t g(\boldsymbol{\theta}, \boldsymbol{c}) + \sigma_t \boldsymbol{\epsilon}, t, \boldsymbol{c}, \boldsymbol{y}) - \boldsymbol{\epsilon}\|_2^2\right], \tag{3}$$

where $\boldsymbol{\epsilon}_{\boldsymbol{\psi}}(\boldsymbol{x}, t, \boldsymbol{c}, \boldsymbol{y})$ is the noise estimator of $\nabla \log q_t(\boldsymbol{x}_t|\boldsymbol{c})$ as in diffusion models. As proposed in ProlificDreamer, $\boldsymbol{\psi}$ is parameterized by LoRA [13] and initialized from a pre-trained diffusion model same as $\nabla \log p_t$.

3. Revealing Mode Collapse in Score Distillation

Despite the remarkable performance of SDS and VSD in 3D asset generation, it is widely observed that the synthesized objects suffer from "*Janus*" artifacts. Janus artifacts refer to the generated 3D scene containing multiple canonical views (the most representative perspective of the object such as the frontal face). In earlier works, Hong et al. [12] and Huang et al. [14] attribute this problem to unimodality of the learned 2D image distribution since the training data for the diffusion models are naturally biased to the most commonly seen views per each category. In this section, we examine extant distillation schemes from a statistical view, which has been overlooked in previous literature.

In principle, natural 2D images can be seen as random projections of 3D scenes. Score distillation matches the image distribution generated by randomly sampled views with a

text-conditioned image distribution to recover the underlying 3D representation. Hence, Janus artifact, in which each view becomes uniform and identical to the most commonly seen views, can be interpreted as a manifestation of distribution collapse to samples within the high-density region. Such distribution degeneration essentially corresponds to the statistical phenomenon *mode collapse*, which happens when an optimized distribution fails to characterize the data diversity and concentrates on a single type of output [1, 7, 25, 36, 49].

Below we theoretically reveal why SDS and VSD are prone to mode collapse. As shown in Poole et al. [31], Wang et al. [56], SDS and VSD equals to the gradient of the following Kullback-Leibler (KL) divergence, i.e., $J_{SDS}(\boldsymbol{\theta}) = J_{VSD}(\boldsymbol{\theta}) = J_{KL}(\boldsymbol{\theta})$ up to an additive constant:

$$J_{KL}(\boldsymbol{\theta}) = \mathbb{E}\left[\Omega(t)\, \mathcal{D}_{\mathrm{KL}}(q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y}) \| p_t(\boldsymbol{x}_t|\boldsymbol{y}))\right], \quad (4)$$

where $\Omega(t) = \omega(t)\sigma_t/\alpha_t$ and the expectation is taken over $t \sim \mathcal{U}[0,T]$ and $\boldsymbol{c} \sim p_c(\boldsymbol{c})$. Here $p_t(\boldsymbol{x}_t|\boldsymbol{y}) = \int p_0(\boldsymbol{x}_0|\boldsymbol{y})\mathcal{N}(\boldsymbol{x}_t|\alpha_t\boldsymbol{x}_0, \sigma_t^2\boldsymbol{I})d\boldsymbol{x}_0$ is the image distribution perturbed by Gaussian noises, while $q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y}) = \int q_0^{\boldsymbol{\theta}}(\boldsymbol{x}_0|\boldsymbol{c})\mathcal{N}(\boldsymbol{x}_t|\alpha_t\boldsymbol{x}_0, \sigma_t^2\boldsymbol{I})d\boldsymbol{x}_0$ models the image distribution generated by 3D parameter $\boldsymbol{\theta}$ with respect to camera pose $\boldsymbol{c}$ and diffused by Gaussian distribution. As shown by Wang et al. [56], $J_{KL}(\boldsymbol{\theta}) = 0$ implies $q_0^{\boldsymbol{\theta}}(\boldsymbol{x}_0|\boldsymbol{c}) = p(\boldsymbol{x}_0|\boldsymbol{y})$, i.e., the distribution of synthesized views satisfy the text-conditioned image distribution.

However, it has not escaped from our notice that $q_0^{\boldsymbol{\theta}}(\boldsymbol{x}_0|\boldsymbol{c}) = \delta(\boldsymbol{x}_0 - g(\boldsymbol{\theta},\boldsymbol{c}))$ is a Dirac distribution for both SDS and VSD. This causes the original KL divergence minimization (Eq. 4) degenerate to a Maximal Likelihood Estimation (MLE) problem:

$$\begin{aligned} J_{KL}(\boldsymbol{\theta}) = &\underbrace{-\mathbb{E}\left[\Omega(t)\, \mathbb{E}_{\boldsymbol{x_t}\sim q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y})} \log p_t(\boldsymbol{x}_t|\boldsymbol{y})\right]}_{J_{MLE}(\boldsymbol{\theta})} \\ &\underbrace{-\mathbb{E}\left[\Omega(t) H[q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y})]\right]}_{const.}, \end{aligned} \quad (5)$$

where $H[q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})] = -\mathbb{E}_{\boldsymbol{x_t}\sim q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y})}[\log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y})]$ denotes the entropy of $q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})$, which turns out to be a constant because $q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y}) = \mathcal{N}(\boldsymbol{x}_t|\alpha_t g(\boldsymbol{\theta},\boldsymbol{c}), \sigma_t^2\boldsymbol{I})$ which has fixed entropy once t, $\boldsymbol{\theta}$ and $\boldsymbol{c}$ have been specified. See full derivation in Appendix A.1.

Note that Eq. 5 signifies $J_{KL}(\boldsymbol{\theta}) = J_{MLE}(\boldsymbol{\theta})$ up to an additive constant, hence $J_{KL}(\boldsymbol{\theta})$ shares all minima with $J_{MLE}(\boldsymbol{\theta})$. It is known that likelihood maximization is more prone to mode collapse. Intuitively, minimizing $J_{MLE}(\boldsymbol{\theta})$ seeks each view *independently* to have the maximum log-likelihood on the image distribution $p(\boldsymbol{x}_0|\boldsymbol{y})$. Since $p(\boldsymbol{x}_0|\boldsymbol{y})$ is usually unimodal and peaks at the canonical view, each view of the scene will collapse to the same local minimum, resulting in Janus artifact (see Fig. 2). We postulate that the existing distillation strategies may be inherently limited by their log-likelihood seeking behaviors, which are more susceptible to mode collapse, especially with biased image distributions.

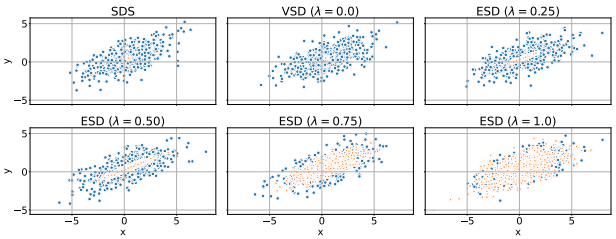

Figure 3. **Gaussian Example.** To illustrate the effects of entropy regularization, we leverage SDS, VSD and ESD to fit a 2D Gaussian distribution. The blue points are sampled from the ground-truth distribution while the orange points are from the fitted distribution.

4. Entropy Regularized Score Distillation

4.1. Entropic Score Distillation

In this section, we highlight the importance of the entropy in score distillation. It is known that higher entropy implies the corresponding distribution could cover a larger support of the ambient space and thus increase the sample diversity. In Eq. 5, the entropy term is shown to diminish in the training objective, which causes each generated view to lack diversity and collapse to a single image with the highest likelihood.

To this end, we propose to bring in an entropy regularization to $J_{MLE}(\boldsymbol{\theta})$ for boosting the view diversity. Since $q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y})$ has constant entropy, we regularize entropy for the distribution $q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y}) = \int q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y})p_c(\boldsymbol{c})d\boldsymbol{c}$, which can be simulated by randomly sampling views from the 3D parameter $\boldsymbol{\theta}$. Consider the following objective:

$$\begin{aligned} J_{Ent}(\boldsymbol{\theta},\lambda) = &-\mathbb{E}\left[\Omega(t)\, \mathbb{E}_{\boldsymbol{x_t}\sim q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y})} \log p_t(\boldsymbol{x}_t|\boldsymbol{y})\right] \\ &-\lambda\, \mathbb{E}\left[\Omega(t) H[q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})]\right], \end{aligned} \quad (7)$$

where λ is a hyper-parameter controlling the regularization strength. We note that without $H[q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})]$, each view is optimized independently and implicitly regularized by the underlying parameterization. However, upon imposing $H[q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})]$, all views become explicitly correlated with each other, as they collectively contribute to the entropy computation. Intuitively, $J_{Ent}(\boldsymbol{\theta},\lambda) = J_{MLE}(\boldsymbol{\theta}) - \lambda\, \mathbb{E}[\Omega(t) H[q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})]]$ seeks the maximal log-likelihood for each view while simultaneously enlarging the entropy for distribution $q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})$, which spans the support and encourages diversity across the rendered views. To gain more insights, we present the following theoretical results:

Theorem 1. *For any $\lambda \in \mathbb{R}$ and $\boldsymbol{\theta} \in \mathbb{R}^D$, we have* $J_{Ent}(\boldsymbol{\theta},\lambda) = \lambda\, \mathbb{E}_t[\Omega(t)\, \mathcal{D}_{\mathrm{KL}}(q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y}) \| p_t(\boldsymbol{x}_t|\boldsymbol{y}))] + (1-\lambda)\, \mathbb{E}_{t,\boldsymbol{c}}[\Omega(t)\, \mathcal{D}_{\mathrm{KL}}(q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y}) \| p_t(\boldsymbol{x}_t|\boldsymbol{y}))] + const.$

Algorithm 1 ESD: Entropic score distillation for text-to-3D generation

Input: A diffusion model $\boldsymbol{\epsilon}_{\phi}(\boldsymbol{x}, t, \boldsymbol{y})$; learnable 3D parameter $\boldsymbol{\theta}$; coefficient λ; text prompt $\boldsymbol{y}$.
Initialize $\boldsymbol{\psi}$ for another diffusion model $\boldsymbol{\epsilon}_{\boldsymbol{\psi}}(\boldsymbol{x}, t, \boldsymbol{y})$ with the parameter $\boldsymbol{\phi}$ specified in diffusion model $\boldsymbol{\epsilon}_{\boldsymbol{\phi}}(\boldsymbol{x}, t, \boldsymbol{y})$, parameterized with LoRA.
while not converged **do**
  Randomly sample a camera pose $\boldsymbol{c} \sim p_c$ and render a view $\boldsymbol{x}_0 = g(\boldsymbol{\theta}, \boldsymbol{c})$ from $\boldsymbol{\theta}$.
  Sample a $t \sim \mathcal{U}[0, T]$ and add Gaussian noise $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$: $\boldsymbol{x}_t = \alpha_t \boldsymbol{x}_0 + \sigma_t \boldsymbol{\epsilon}$.
  $$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta_1 \left[\omega(t) \frac{\partial g(\boldsymbol{\theta}, \boldsymbol{c})}{\partial \boldsymbol{\theta}} (\boldsymbol{\epsilon}_{\boldsymbol{\phi}}(\boldsymbol{x}_t, t, \boldsymbol{y}) - \lambda \boldsymbol{\epsilon}_{\boldsymbol{\psi}}(\boldsymbol{x}_t, t, \boldsymbol{\emptyset}, \boldsymbol{y}) - (1-\lambda) \boldsymbol{\epsilon}_{\boldsymbol{\psi}}(\boldsymbol{x}_t, t, \boldsymbol{c}, \boldsymbol{y})\right] \tag{6}$$
  With probability $1 - p_{\emptyset}$, $\boldsymbol{\psi} \leftarrow \boldsymbol{\psi} - \eta_2 \nabla_{\boldsymbol{\psi}} \left[\omega(t) \|\boldsymbol{\epsilon}_{\boldsymbol{\psi}}(\boldsymbol{x}_t, t, \boldsymbol{c}, \boldsymbol{y}) - \boldsymbol{\epsilon}\|_2^2\right]$.
  Otherwise, $\boldsymbol{\psi} \leftarrow \boldsymbol{\psi} - \eta_2 \nabla_{\boldsymbol{\psi}} \left[\omega(t) \|\boldsymbol{\epsilon}_{\boldsymbol{\psi}}(\boldsymbol{x}_t, t, \boldsymbol{\emptyset}, \boldsymbol{y}) - \boldsymbol{\epsilon}\|_2^2\right]$.
end while
Return $\boldsymbol{\theta}$

We prove Theorem 1 in Appendix A.3. Theorem 1 implies that $J_{Ent}(\boldsymbol{\theta}, \lambda)$ essentially equal to a combination of two types of KL divergences, where the former one minimizes the distribution discrepancy between $q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})$ and $p_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})$ which marginalizes the camera pose within $q_t^{\boldsymbol{\theta}}$, while the latter is the original KL divergence $J_{KL}(\boldsymbol{\theta})$ adopted by SDS and VSD which takes expectation over $\boldsymbol{c}$ out of KL divergence.

Next, we derive the gradient of $J_{Ent}(\boldsymbol{\theta}, \lambda)$ that will be backpropagated to update the 3D representation. It can be obtained by path derivative and reparameterization trick:

$$\nabla_{\boldsymbol{\theta}} J_{Ent}(\boldsymbol{\theta}, \lambda) = -\mathbb{E}\left[\omega(t) \frac{\partial g(\boldsymbol{\theta}, \boldsymbol{c})}{\partial \boldsymbol{\theta}} \left(\sigma_t \nabla \log p_t(\boldsymbol{x}_t|\boldsymbol{y}) - \lambda \sigma_t \nabla \log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})\right)\right]. \tag{8}$$

The full derivation is deferred to Appendix A.2. We name this update rule as *Entropic Score Distillation (ESD)*. Note that ESD differs from VSD as its second score function does not depend on the camera pose.

4.2. Classifier-Free Guidance Trick

Similar to SDS and VSD, we approximate $\nabla \log p_t(\boldsymbol{x}_t|\boldsymbol{y})$ via a pre-trained diffusion model $\boldsymbol{\epsilon}_{\boldsymbol{\phi}}(\boldsymbol{x}_t, t, \boldsymbol{y})$. However, $\nabla \log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{y})$ is not readily available. We found that directly fine-tuning a pre-trained diffusion model using rendered images to approximate $\nabla \log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{y})$, akin to ProlificDreamer, does not yield robust performance. We postulate this difficulty arises from the removal of the camera condition, increasing the complexity of the distribution to be fitted.

To tackle this problem, we recall the result in Theorem 1 that $J_{Ent}(\boldsymbol{\theta}, \lambda)$ can be written in terms of two KL divergence losses. Therefore, its gradient can be decomposed as a weighted combination of their gradients, which correspond to unconditional and conditional score functions in terms of the camera pose $\boldsymbol{c}$, respectively:

$$\nabla_{\boldsymbol{\theta}} J_{Ent}(\boldsymbol{\theta}, \lambda) = -\mathbb{E}\left[\omega(t) \frac{\partial g(\boldsymbol{\theta}, \boldsymbol{c})}{\partial \boldsymbol{\theta}} (\sigma_t \nabla \log p_t(\boldsymbol{x}_t|\boldsymbol{y}) - \lambda \sigma_t \nabla \log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y}) - (1-\lambda) \sigma_t \nabla \log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c}, \boldsymbol{y}))\right]. \tag{9}$$

We formally prove Eq. 9 in Appendix A.3. With the above formulation, ESD can be implemented via the Classifier-Free Guidance (CFG) trick, which was initially proposed to balance the variety and quality of text-conditionally generated images from diffusion models [10]. Algorithm 1 outlines the computation paradigm of ESD, in which we surrogate score functions in Eq. 9 with pre-trained and fine-tuned diffusion models (see Eq. 6), and takes random turns with a probability $p_{\emptyset}$ to balance the training of conditional and unconditional score functions, as suggested by Ho and Salimans [10].

4.3. Discussion

In VSD, the camera-conditioned score is believed to play a significant role in facilitating visual quality. Intuitively, such conditioning can equip the tuned diffusion model with multi-view priors [20]. Also, Hertz et al. [8] suggests such a method can be useful to stabilize the update of the implicit parameters. However, ESD counters this argument by suggesting that the camera condition might not always be advantageous, particularly when the particle size is reduced to one. In such cases, the resulting KL divergence provably degenerates to a likelihood maximization algorithm vulnerable to mode collapse.

It is noteworthy that, even though their subtle differences in implementation, the optimization objectives of ESD and VSD are fundamentally different (see Sec. 4.1). ESD sets itself apart from VSD by incorporating entropy regularization, a crucial feature absent in VSD, aiming to augment diversity across views. Despite originating from distinct objectives, our theoretical establishment allows for a straightforward implementation of ESD based on VSD using the CFG trick.

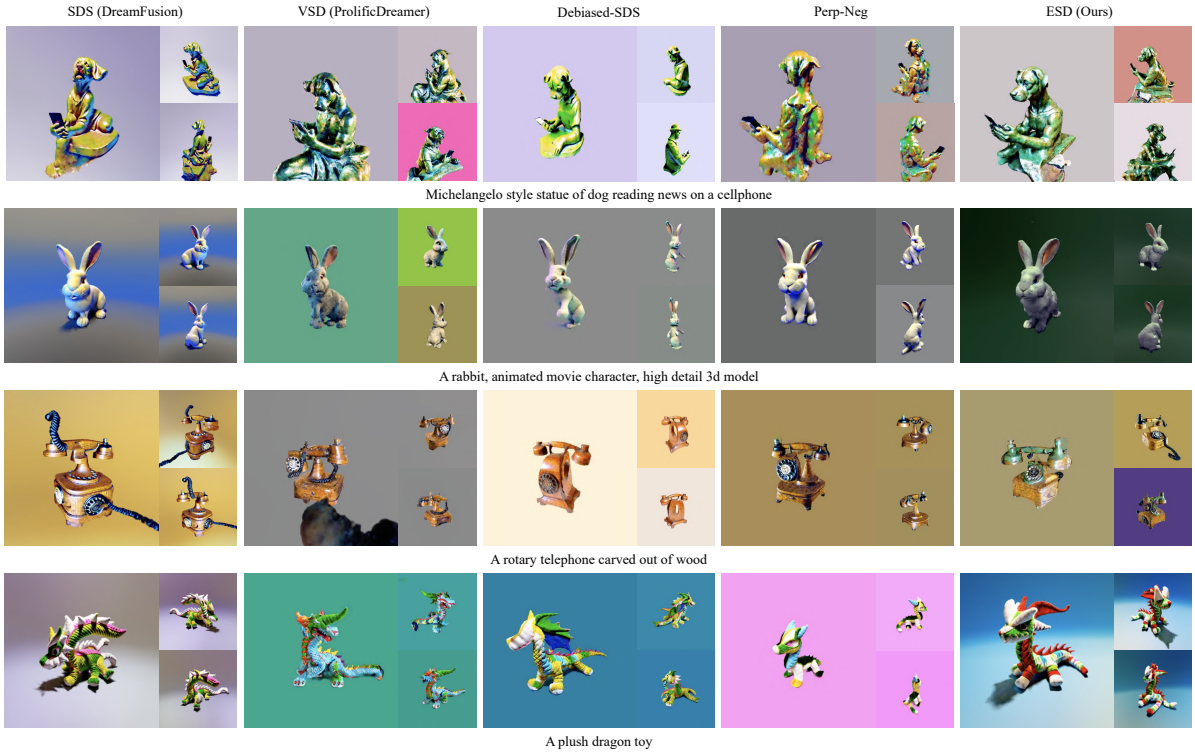

Figure 4. **Qualitative Results.** Our proposed outperforms all baselines in terms of better geometry and well-constructed texture details. Our results deliver photo-realistic and diverse rendered views, while baseline methods more or less suffer from the Janus problem. Best view in an electronic copy.

We provide an illustrative example by leveraging SDS, VSD and ESD (with different λ's) to fit a 2D Gaussian distribution in Fig. 3. With SDS and VSD, all samples are converged to the high-density area while ESD recovers the entire support of the distribution. We provide more details and examples in Appendix B.

5. Other Related Work

Text-to-Image Diffusion Model. Text-to-image diffusion models [32, 33] are cornerstone components of text-to-3D generation. It involves text embedding conditioning into the iterative denoising process. Equipped with large-scale image-text paired datasets, many works [29, 33, 35] scale up to tackle text-to-image generation. Among them, latent diffusion models attracted great interest in the open-source community since they reduced the computation cost by diffusing in the low-resolution latent space instead of directly in the pixel space. In addition, text-to-image diffusion models have also found applications in various computer vision tasks, including text-to-3D [31, 43], image-to-3D [59], text-to-svg [17], text-to-video [18, 42], etc.

3D Generation with 2D Priors. Well-annotated 3D data requires immense effort to collect. Instead, a line of research studies on how to learn 3D generative models using 2D supervision. Early attempts, including pi-GAN [34], EG3D [3], GRAF [37], GIRAFFE [30], adopt adversarial loss between the rendered images and natural images. DreamField [16] leverages CLIP to align NeRF with text prompts. More recently, with the rapid development of text-to-image diffusion models, diffusion-based image priors have attracted increasing interest, and score distillation has then become the dominant technique. Pioneer works DreamFusion [31] and ProlificDreamer [56] have been introduced in detail in Sec. 2. Their concurrent work SJC [54] derives the score Jacobian chaining method from another theoretical viewpoint of Perturb and Average Scoring. Even though diffusion models directly trained with 3D data nowadays demonstrate largely improved results [21, 41], score distillation still plays a pivotal role in ensuring view consistency.

Techniques to Improve Score Distillation. Providing the empirical promise of score distillation, there have been numerous techniques proposed to improve its effectiveness. Magic3D [19] and Fantasia3D [4] utilize mesh and DMTet [40] to disentangle the optimization of geometry and texture. TextMesh [50] and 3DFuse [38] use depth-conditioned text-to-image diffusion priors that support geometry-aware texturing. Score debiasing[12] and Perp-Neg [2] study to refine the text prompts for a better 3D generation. DreamTime [14] and RED-Diff [24] investigate the timestep scheduling in the

Figure 5. **Qualitative Results.** We combine our proposed ESD with timestep scheduling in DreamTime [14] and compare it against baseline methods. Prompt: A caramic lion.

score distillation process. HIFA [60] adopts multiple diffusion steps for distillation. Score distillation also works with auxiliary losses, including CLIP loss [59] and adversarial loss [5, 39].

6. Evaluation Metrics

In this section, we introduce four information-theoretic metrics to numerically evaluate the generated 3D results with a particular focus on identifying Janus artifacts or mode collapse. The metrics we propose comprehensively cover four aspects: 1) the relevance with the text prompts, 2) distribution fitness, 3) rendering quality, and 4) view diversity.

CLIP Distance. We compute the average distance between rendered images and the text embedding to reflect the relevance between generated results and the specified text prompt. Specifically, we render N views from the generated 3D representations, and for each view, we obtain an embedding vector through the image encoder of a CLIP model [53]. In the meantime, we compute the text embedding utilizing the text encoder. The CLIP distance is computed as the one minus cosine similarity between the image embeddings and text embeddings averaged over all views.

Fréchet inception distance (FID). As shown in Sec. 3 and 4, score distillation essentially matches distributions via KL divergence. Hence, it becomes reasonable to employ FID to measure the distance between the image distribution $q^{\boldsymbol{\theta}}(\boldsymbol{x}_0|\boldsymbol{y})$ generated by randomly rendering 3D representation and the text-conditioned image distribution $p(\boldsymbol{x}_0|\boldsymbol{y})$ modeled by a diffusion model. We sample N images using pre-trained latent diffusion model given text prompts as the ground truth image dataset, and render N views uniformly distributed over a unit sphere from the optimized 3D scene as the generated image dataset. Then standard FID [9] is computed between these two sets of images. Note that FID is known to be effective in quantitatively identifying mode collapse.

Inception Quality and Variety. Thanks to our established connection with mode collapse, we know that Janus problem is due to a lack of sample diversity. Inspired by Inception Score (IS) [36], we utilize entropy-related metrics to reflect the generated image quality and diversity. We propose Inception Quality (IQ) and Inception Variety (IV), formulated as below:

$$IQ(\boldsymbol{\theta}) = \mathbb{E}_{\boldsymbol{c}}\left[H[p_{cls}(\boldsymbol{y}|g(\boldsymbol{\theta},\boldsymbol{c}))]\right], \tag{10}$$

$$IV(\boldsymbol{\theta}) = H[\mathbb{E}_{\boldsymbol{c}}[p_{cls}(\boldsymbol{y}|g(\boldsymbol{\theta},\boldsymbol{c})]], \tag{11}$$

where $p_{cls}(\boldsymbol{y}|\boldsymbol{x})$ is a pre-trained classifier. IQ computes the average entropy of the label logits predicted for all rendered views, while IV computes the entropy of the averaged label logits of all rendered views. Intuitively, the smaller IQ means highly confident classification results on rendered views, which also indicates better visual quality of generated 3D assets. In the meanwhile, the higher IV signifies that each rendered view is likely to have a distinct label prediction, meaning the 3D creation has higher view diversity. Note that IV upper bounds IQ due to Jensen inequality. So we can define Inception Gain $IG = (IV - IQ)/IQ$, which characterizes the information gain brought by knowing where the camera pose is, namely the improvement of distinguishability among different views.

7. Experiments

Settings. In this section, we empirically validate the effectiveness of our proposal. The chosen prompts are targeted at objects with clearly defined canonical views, posing a challenge for existing methods. Our baseline approaches include SDS (DreamFusion) [31] and VSD (ProlificDreamer) [56], as well as two methods dedicated to solving Janus problem: Debiased-SDS [12] and Perp-Neg [2]. For fair comparison, all experiments are benchmarked under the open-source threestudio framework. Geometry refinement [56] is adopted for all distillation schemes. Please refer to Appendix C for more implementation details.

Figure 6. **Ablation Studies on λ.** We investigate the choice of different entropy regularization strength λ. Prompt: Michelangelo-style statue of dog reading news on a cellphone.

Table 1. **Quantitative Comparisons.** (↓) means the lower the better, and (↑) means the higher the better.

	CLIP (↓)	FID (↓)	IQ (↓)	IV (↑)	IG (↑)	SR (↑)
SDS	0.737	291.860	4.295	**4.8552**	0.123	15.00%
VSD	0.725	265.141	3.149	3.5712	0.137	19.17%
ESD	**0.714**	**235.915**	**3.135**	4.0314	**0.327**	**55.83%**

Qualitative Comparison. We present qualitative comparisons in Fig. 4. We encourage interested readers to Appendix D for more results and our project page for videos. It is clearly shown that our proposed ESD delivers more precise geometry with the Janus problem alleviated. In comparison, the results presented by SDS and VSD all contain more or less corrupted geometry with multi-face structures. Debiased-SDS and Perp-Neg are shown to be effective for some text prompts, while not so consistent as ESD. Additionally, we find that ESD can work particularly well when combined with the time-prioritized scheduling proposed in DreamTime [14], as shown in Fig. 5. This means ESD is orthogonal to many other methods and can cooperate with them to further reduce Janus artifacts.

Quantitative Comparison. With metrics proposed in Sec. 6, we numerically evaluate our method and baselines across 120 text prompts provided in [58]. We additionally involve Successful generation Rate (SR) based on human evaluation. The results are presented in Tab. 1. We observe that among all metrics, ESD reaches the best CLIP score, FID, and IG. More importantly, ESD achieves the optimal balance between view quality and diversity as shown by IQ and IV. Whereas, SDS suffers from low image quality with high IQ and VSD is limited by insufficient view variety with low IV. The superior IG of ESD indicates that views inside the generated scene are distinguishable rather than collapsing to be the same. We defer the breakdown table for numerical evaluation on examples in Fig. 4, human evaluation criteria, and the standard deviation of metrics to Appendix E.

Ablation Studies We conduct ablation studies on the choice of λ (*i.e.* CFG weights) in Fig. 6. We demonstrate that λ can adjust ESD's preference toward view- quality or diversity. When set to one, the produced Janus-free result albeits with fewer realistic details in the textures. Conversely, when set to zero, ESD equates to VSD, and the Janus problem emerges again. We empirically find that choosing λ around 0.5 yields the best result, balancing fine-grained textures and well-constructed geometry. We also implement ESD by directly fitting the score function $\nabla \log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})$ without camera pose conditioning to validate the suggested implementation by CFG trick. We show in Fig. 7 that this optimization scheme is unstable. As training proceeds, the gradient explodes, and the optimized texture overflows.

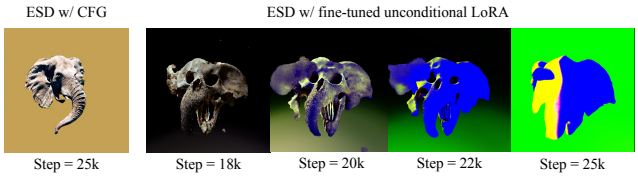

Figure 7. **Ablation on Implementations.** The successfully generated result is obtained via our suggested CFG trick while the diverged result is yielded by fitting the unconditioned score function in Eq. 8 via LoRA. Prompt: an elephant skull.

8. Conclusion

In this paper, we reveal that existing score distillation methods degenerate to maximal likelihood seeking on each view independently, leading to the mode collapse problem. We identify that re-establishing the entropy term in the variational objective brings a new update rule, called Entropic Score Distillation (ESD), which is theoretically equivalent to adopting classifier-free guidance trick upon variational score distillation. ESD maximizes the entropy of the rendered image distribution, encouraging diversity across views and mitigating the Janus problem.

Acknowledgments

P Wang is sincerely grateful for constructive feedback regarding this manuscript from Zhaoyang Lv, Xiaoyu Xiang, Amit Kumar, Jinhui Xiong, and Varun Nagaraja. P Wang also thanks Ruisi Cai for providing decent visual materials for illustration purposes. Any statements, opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of their employers or the supporting entities.

References

[1] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In *International conference on machine learning*, pages 214–223. PMLR, 2017. 4

[2] Mohammadreza Armandpour, Huangjie Zheng, Ali Sadeghian, Amir Sadeghian, and Mingyuan Zhou. Re-imagine the negative prompt algorithm: Transform 2d diffusion into 3d, alleviate janus problem and beyond. *arXiv preprint arXiv:2304.04968*, 2023. 2, 6, 7, 8

[3] Eric R Chan, Connor Z Lin, Matthew A Chan, Koki Nagano, Boxiao Pan, Shalini De Mello, Orazio Gallo, Leonidas J Guibas, Jonathan Tremblay, Sameh Khamis, et al. Efficient geometry-aware 3d generative adversarial networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16123–16133, 2022. 6

[4] Rui Chen, Yongwei Chen, Ningxin Jiao, and Kui Jia. Fantasia3d: Disentangling geometry and appearance for high-quality text-to-3d content creation. *arXiv preprint arXiv:2303.13873*, 2023. 1, 3, 6

[5] Yiwen Chen, Chi Zhang, Xiaofeng Yang, Zhongang Cai, Gang Yu, Lei Yang, and Guosheng Lin. It3d: Improved text-to-3d generation with explicit view synthesis. *arXiv preprint arXiv:2308.11473*, 2023. 7

[6] Giannis Daras, Kulin Shah, Yuval Dagan, Aravind Gollakota, Alex Dimakis, and Adam Klivans. Ambient diffusion: Learning clean distributions from corrupted data. *Advances in Neural Information Processing Systems*, 36, 2024. 6

[7] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. *Advances in neural information processing systems*, 27, 2014. 4

[8] Amir Hertz, Kfir Aberman, and Daniel Cohen-Or. Delta denoising score. *arXiv preprint arXiv:2304.07090*, 2023. 5

[9] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. *Advances in neural information processing systems*, 30, 2017. 7

[10] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022. 2, 5

[11] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020. 2

[12] Susung Hong, Donghoon Ahn, and Seungryong Kim. Debiasing scores and prompts of 2d diffusion for robust text-to-3d generation. *arXiv preprint arXiv:2303.15413*, 2023. 1, 2, 3, 6, 7, 8

[13] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021. 3

[14] Yukun Huang, Jianan Wang, Yukai Shi, Xianbiao Qi, Zheng-Jun Zha, and Lei Zhang. Dreamtime: An improved optimization strategy for text-to-3d content creation. *arXiv preprint arXiv:2306.12422*, 2023. 3, 6, 7, 8

[15] Aapo Hyvärinen and Peter Dayan. Estimation of non-normalized statistical models by score matching. *Journal of Machine Learning Research*, 6(4), 2005. 3

[16] Ajay Jain, Ben Mildenhall, Jonathan T Barron, Pieter Abbeel, and Ben Poole. Zero-shot text-guided object generation with dream fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 867–876, 2022. 6

[17] Ajay Jain, Amber Xie, and Pieter Abbeel. Vectorfusion: Text-to-svg by abstracting pixel-based diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1911–1920, 2023. 6

[18] Levon Khachatryan, Andranik Movsisyan, Vahram Tadevosyan, Roberto Henschel, Zhangyang Wang, Shant Navasardyan, and Humphrey Shi. Text2video-zero: Text-to-image diffusion models are zero-shot video generators. *arXiv preprint arXiv:2303.13439*, 2023. 6

[19] Chen-Hsuan Lin, Jun Gao, Luming Tang, Towaki Takikawa, Xiaohui Zeng, Xun Huang, Karsten Kreis, Sanja Fidler, Ming-Yu Liu, and Tsung-Yi Lin. Magic3d: High-resolution text-to-3d content creation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 300–309, 2023. 1, 3, 6

[20] Ruoshi Liu, Rundi Wu, Basile Van Hoorick, Pavel Tokmakov, Sergey Zakharov, and Carl Vondrick. Zero-1-to-3: Zero-shot one image to 3d object. *arXiv preprint arXiv:2303.11328*, 2023. 5

[21] Yuan Liu, Cheng Lin, Zijiao Zeng, Xiaoxiao Long, Lingjie Liu, Taku Komura, and Wenping Wang. Syncdreamer: Generating multiview-consistent images from a single-view image. *arXiv preprint arXiv:2309.03453*, 2023. 6

[22] Ziming Liu, Di Luo, Yilun Xu, Tommi Jaakkola, and Max Tegmark. Genphys: From physical processes to generative models. *arXiv preprint arXiv:2304.02637*, 2023. 3

[23] Jonathan Lorraine, Kevin Xie, Xiaohui Zeng, Chen-Hsuan Lin, Towaki Takikawa, Nicholas Sharp, Tsung-Yi Lin, Ming-Yu Liu, Sanja Fidler, and James Lucas. Att3d: Amortized text-to-3d object synthesis. *arXiv preprint arXiv:2306.07349*, 2023. 8

[24] Morteza Mardani, Jiaming Song, Jan Kautz, and Arash Vahdat. A variational perspective on solving inverse problems with diffusion models. *arXiv preprint arXiv:2305.04391*, 2023. 6

[25] Luke Metz, Ben Poole, David Pfau, and Jascha Sohl-Dickstein. Unrolled generative adversarial networks. *arXiv preprint arXiv:1611.02163*, 2016. 4

[26] Gal Metzer, Elad Richardson, Or Patashnik, Raja Giryes, and Daniel Cohen-Or. Latent-nerf for shape-guided generation of 3d shapes and textures. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12663–12673, 2023. 1, 3

[27] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. In *European conference on computer vision*, pages 405–421. Springer, 2020. 1, 3

[28] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. *arXiv preprint arXiv:2201.05989*, 2022. 6

[29] Alex Nichol, Prafulla Dhariwal, Aditya Ramesh, Pranav Shyam, Pamela Mishkin, Bob McGrew, Ilya Sutskever, and Mark Chen. Glide: Towards photorealistic image generation and editing with text-guided diffusion models. *arXiv preprint arXiv:2112.10741*, 2021. 6

[30] Michael Niemeyer and Andreas Geiger. Giraffe: Representing scenes as compositional generative neural feature fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11453–11464, 2021. 6

[31] Ben Poole, Ajay Jain, Jonathan T Barron, and Ben Mildenhall. Dreamfusion: Text-to-3d using 2d diffusion. *arXiv preprint arXiv:2209.14988*, 2022. 1, 2, 3, 4, 6, 7, 8

[32] Aditya Ramesh, Mikhail Pavlov, Gabriel Goh, Scott Gray, Chelsea Voss, Alec Radford, Mark Chen, and Ilya Sutskever. Zero-shot text-to-image generation. In *International Conference on Machine Learning*, pages 8821–8831. PMLR, 2021. 6

[33] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. Hierarchical text-conditional image generation with clip latents. *arXiv preprint arXiv:2204.06125*, 2022. 6

[34] René Ranftl, Alexey Bochkovskiy, and Vladlen Koltun. Vision transformers for dense prediction. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 12179–12188, 2021. 6

[35] Chitwan Saharia, William Chan, Saurabh Saxena, Lala Li, Jay Whang, Emily Denton, Seyed Kamyar Seyed Ghasemipour, Raphael Gontijo-Lopes, Burcu Karagol Ayan, Tim Salimans, Jonathan Ho, David J. Fleet, and Mohammad Norouzi. Photorealistic text-to-image diffusion models with deep language understanding. In *Advances in Neural Information Processing Systems*, 2022. 6

[36] Tim Salimans, Ian Goodfellow, Wojciech Zaremba, Vicki Cheung, Alec Radford, and Xi Chen. Improved techniques for training gans. *Advances in neural information processing systems*, 29, 2016. 2, 4, 7, 8

[37] Katja Schwarz, Yiyi Liao, Michael Niemeyer, and Andreas Geiger. Graf: Generative radiance fields for 3d-aware image synthesis. *Advances in Neural Information Processing Systems*, 33:20154–20166, 2020. 6

[38] Junyoung Seo, Wooseok Jang, Min-Seop Kwak, Jaehoon Ko, Hyeonsu Kim, Junho Kim, Jin-Hwa Kim, Jiyoung Lee, and Seungryong Kim. Let 2d diffusion model know 3d-consistency for robust text-to-3d generation. *arXiv preprint arXiv:2303.07937*, 2023. 6

[39] Ruizhi Shao, Jingxiang Sun, Cheng Peng, Zerong Zheng, Boyao Zhou, Hongwen Zhang, and Yebin Liu. Control4d: Dynamic portrait editing by learning 4d gan from 2d diffusion-based editor. *arXiv preprint arXiv:2305.20082*, 2023. 7

[40] Tianchang Shen, Jun Gao, Kangxue Yin, Ming-Yu Liu, and Sanja Fidler. Deep marching tetrahedra: a hybrid representation for high-resolution 3d shape synthesis. *Advances in Neural Information Processing Systems*, 34:6087–6101, 2021. 6

[41] Yichun Shi, Peng Wang, Jianglong Ye, Mai Long, Kejie Li, and Xiao Yang. Mvdream: Multi-view diffusion for 3d generation. *arXiv preprint arXiv:2308.16512*, 2023. 6

[42] Uriel Singer, Adam Polyak, Thomas Hayes, Xi Yin, Jie An, Songyang Zhang, Qiyuan Hu, Harry Yang, Oron Ashual, Oran Gafni, et al. Make-a-video: Text-to-video generation without text-video data. *arXiv preprint arXiv:2209.14792*, 2022. 6

[43] Uriel Singer, Shelly Sheynin, Adam Polyak, Oron Ashual, Iurii Makarov, Filippos Kokkinos, Naman Goyal, Andrea Vedaldi, Devi Parikh, Justin Johnson, et al. Text-to-4d dynamic scene generation. *arXiv preprint arXiv:2301.11280*, 2023. 6

[44] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International Conference on Machine Learning*, pages 2256–2265. PMLR, 2015. 2

[45] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*, 2020. 3

[46] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. *Advances in neural information processing systems*, 32, 2019. 2

[47] Yang Song, Sahaj Garg, Jiaxin Shi, and Stefano Ermon. Sliced score matching: A scalable approach to density and score estimation. In *Uncertainty in Artificial Intelligence*, pages 574–584. PMLR, 2020. 3

[48] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020. 2, 3

[49] Akash Srivastava, Lazar Valkov, Chris Russell, Michael U Gutmann, and Charles Sutton. Veegan: Reducing mode collapse in gans using implicit variational learning. *Advances in neural information processing systems*, 30, 2017. 4

[50] Christina Tsalicoglou, Fabian Manhardt, Alessio Tonioni, Michael Niemeyer, and Federico Tombari. Textmesh: Generation of realistic 3d meshes from text prompts. *arXiv preprint arXiv:2304.12439*, 2023. 1, 3, 6

[51] Cédric Villani et al. *Optimal transport: old and new*. Springer, 2009. 3

[52] Pascal Vincent. A connection between score matching and denoising autoencoders. *Neural computation*, 23(7):1661–1674, 2011. 3

[53] Can Wang, Menglei Chai, Mingming He, Dongdong Chen, and Jing Liao. Clip-nerf: Text-and-image driven manipulation of neural radiance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3835–3844, 2022. 7

[54] Haochen Wang, Xiaodan Du, Jiahao Li, Raymond A Yeh, and Greg Shakhnarovich. Score jacobian chaining: Lifting pretrained 2d diffusion models for 3d generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12619–12629, 2023. 1, 3, 6

[55] Peihao Wang, Zhiwen Fan, Dejia Xu, Dilin Wang, Sreyas Mohan, Forrest Iandola, Rakesh Ranjan, Yilei Li, Qiang Liu,

Zhangyang Wang, and Vikas Chandra. Steindreamer: Variance reduction for text-to-3d score distillation via stein identity. *arXiv preprint arXiv:2401.00604*, 2023.

[56] Zhengyi Wang, Cheng Lu, Yikai Wang, Fan Bao, Chongxuan Li, Hang Su, and Jun Zhu. Prolificdreamer: High-fidelity and diverse text-to-3d generation with variational score distillation. *arXiv preprint arXiv:2305.16213*, 2023. 1, 2, 3, 4, 6, 7, 8

[57] Zhendong Wang, Yifan Jiang, Huangjie Zheng, Peihao Wang, Pengcheng He, Zhangyang Wang, Weizhu Chen, and Mingyuan Zhou. Patch diffusion: Faster and more data-efficient training of diffusion models. *Advances in Neural Information Processing Systems*, 36, 2024. 6

[58] Tong Wu, Guandao Yang, Zhibing Li, Kai Zhang, Ziwei Liu, Leonidas Guibas, Dahua Lin, and Gordon Wetzstein. Gpt-4v (ision) is a human-aligned evaluator for text-to-3d generation. *arXiv preprint arXiv:2401.04092*, 2024. 8, 7

[59] Dejia Xu, Yifan Jiang, Peihao Wang, Zhiwen Fan, Yi Wang, and Zhangyang Wang. Neurallift-360: Lifting an in-the-wild 2d photo to a 3d object with 360 {\deg} views. *arXiv preprint arXiv:2211.16431*, 2022. 3, 6, 7

[60] Joseph Zhu and Peiye Zhuang. Hifa: High-fidelity text-to-3d with advanced diffusion guidance. *arXiv preprint arXiv:2305.18766*, 2023. 7

Taming Mode Collapse in Score Distillation for Text-to-3D Generation

Supplementary Material

A. Deferred Theory

We present deferred proofs and derivations in this section. In the beginning, we justify several claimed properties of J_{KL} (Eq. 4). Then we formally derive ESD (Eq. 8) via our proposed objective J_{Ent} (Eq. 7). Lastly, we prove that Classifier-Free Guidance trick (CFG) (Eq. 9) can be used to implement ESD.

A.1. Justification of Vanilla KL Divergence J_{KL}

Let us consider KL divergence objective restated from Eq. 4:

$$J_{KL}(\boldsymbol{\theta}) = \mathbb{E}_{t\sim\mathcal{U}[0,T],\boldsymbol{c}\sim p_c(\boldsymbol{c})}\left[\omega(t)\frac{\sigma_t}{\alpha_t}\,\mathcal{D}_{\mathrm{KL}}(q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y})\|p_t(\boldsymbol{x}_t|\boldsymbol{y}))\right], \tag{12}$$

where we recall the notations: $\alpha_t, \sigma_t \in \mathbb{R}_+$ are time-dependent diffusion coefficients, $\boldsymbol{c} \sim p_c(\boldsymbol{c})$ is a camera pose drawn from a prior distribution over $\mathbb{SO}(3) \times \mathbb{R}^3$, and $g(\boldsymbol{\theta}, \boldsymbol{c})$ renders an image at viewpoint $\boldsymbol{c}$ from the 3D representation $\boldsymbol{\theta}$. $p_t(\boldsymbol{x}_t|\boldsymbol{y})$ is the Gaussian diffused image distribution denoted as below:

$$p_t(\boldsymbol{x}_t|\boldsymbol{y}) = \int p_0(\boldsymbol{x}_0|\boldsymbol{y})\,\mathcal{N}(\boldsymbol{x}_t|\alpha_t\boldsymbol{x}_0, \sigma_t^2\boldsymbol{I})d\boldsymbol{x}_0, \tag{13}$$

where $p_0(\boldsymbol{x}_0|\boldsymbol{y})$ is the text-conditioned distribution of clean images. We also define $q_t(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y})$ as the Gaussian diffused distribution of rendered images:

$$q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y}) = \int q_0^{\boldsymbol{\theta}}(\boldsymbol{x}_0|\boldsymbol{c})\,\mathcal{N}(\boldsymbol{x}_t|\alpha_t\boldsymbol{x}_0, \sigma_t^2\boldsymbol{I})d\boldsymbol{x}_0, \tag{14}$$

where we assume $\boldsymbol{x}_0$ is independent of text prompt $\boldsymbol{y}$ given the camera pose and underlying 3D representation. Furthermore, we assume the rendering process has no randomness, thus $q_0^{\boldsymbol{\theta}}(\boldsymbol{x}_0|\boldsymbol{c}) = \delta(\boldsymbol{x}_0 - g(\boldsymbol{\theta}, \boldsymbol{c}))$ can be written as a Dirac distribution.

Now, we can derive the gradient of $J_{KL}(\boldsymbol{\theta})$, as summarized in the following lemma:

Lemma 1 (Gradient of J_{KL})**.** *For any $\boldsymbol{\theta}$, we have:*

$$\nabla_{\boldsymbol{\theta}} J_{KL}(\boldsymbol{\theta}) = -\,\mathbb{E}_{t\sim\mathcal{U}[0,T],\boldsymbol{c}\sim p_c(\boldsymbol{c}),\boldsymbol{\epsilon}\sim\mathcal{N}(\boldsymbol{0},\boldsymbol{I})}\left[\omega(t)\frac{\partial g(\boldsymbol{\theta},\boldsymbol{c})}{\partial\boldsymbol{\theta}}\sigma_t\nabla\log p_t(\boldsymbol{x}_t|\boldsymbol{y})\right], \tag{15}$$

where $\boldsymbol{x}_t = \alpha_t\boldsymbol{x}_0 + \sigma_t\boldsymbol{\epsilon}$, and $\boldsymbol{x}_0 = g(\boldsymbol{\theta}, \boldsymbol{c})$.

Proof. Due to the linearity of expectation, we have:

$$\nabla_{\boldsymbol{\theta}}\,\mathbb{E}_{t\sim\mathcal{U}[0,T],\boldsymbol{c}\sim p_c(\boldsymbol{c})}\left[\omega(t)\frac{\sigma_t}{\alpha_t}\,\mathcal{D}_{\mathrm{KL}}(q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y})\|p_t(\boldsymbol{x}_t|\boldsymbol{y}))\right] \tag{16}$$

$$= \mathbb{E}_{t\sim\mathcal{U}[0,T],\boldsymbol{c}\sim p_c(\boldsymbol{c})}\left[\omega(t)\frac{\sigma_t}{\alpha_t}\nabla_{\theta}\,\mathcal{D}_{\mathrm{KL}}(q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y})\|p_t(\boldsymbol{x}_t|\boldsymbol{y}))\right] \tag{17}$$

$$= \mathbb{E}_{t\sim\mathcal{U}[0,T],\boldsymbol{c}\sim p_c(\boldsymbol{c})}\left[\omega(t)\frac{\sigma_t}{\alpha_t}\nabla_{\boldsymbol{\theta}}\,\mathbb{E}_{\boldsymbol{x}_t\sim q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y})}\left[\log\frac{q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y})}{p_t(\boldsymbol{x}_t|\boldsymbol{y})}\right]\right] \tag{18}$$

Fixing t and $\boldsymbol{c}$, we apply reparameterization trick:

$$\nabla_{\boldsymbol{\theta}}\,\mathbb{E}_{\boldsymbol{x}_t\sim q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y})}\left[\log\frac{q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y})}{p_t(\boldsymbol{x}_t|\boldsymbol{y})}\right] = \mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(\boldsymbol{0},\boldsymbol{I})}\left[\underbrace{\nabla_{\boldsymbol{\theta}}\log q_t^{\boldsymbol{\theta}}(\alpha_t g(\boldsymbol{\theta},\boldsymbol{c}) + \sigma_t\boldsymbol{\epsilon}|\boldsymbol{c},\boldsymbol{y})}_{(a)} - \underbrace{\nabla_{\boldsymbol{\theta}}\log p_t(\alpha_t g(\boldsymbol{\theta},\boldsymbol{c}) + \sigma_t\boldsymbol{\epsilon}|\boldsymbol{y})}_{(b)}\right]. \tag{19}$$

Notice that $q_t^{\boldsymbol{\theta}}(\alpha_t g(\boldsymbol{\theta}, \boldsymbol{c}) + \sigma_t \boldsymbol{\epsilon} | \boldsymbol{c}, \boldsymbol{y}) = \mathcal{N}(\boldsymbol{\epsilon}|\mathbf{0}, \boldsymbol{I})$ by substituting to Eq. 14, which is independent of $\boldsymbol{\theta}$. Thus $(a) = \mathbf{0}$. For term (b), by chain rule, we have:

$$\nabla_{\boldsymbol{\theta}} \log p_t(\alpha_t g(\boldsymbol{\theta}, \boldsymbol{c}) + \sigma_t \boldsymbol{\epsilon} | \boldsymbol{y}) = \alpha_t \frac{\partial g(\boldsymbol{\theta}, \boldsymbol{c})}{\partial \boldsymbol{\theta}} \nabla \log p_t(\alpha_t g(\boldsymbol{\theta}, \boldsymbol{c}) + \sigma_t \boldsymbol{\epsilon} | \boldsymbol{y}). \tag{20}$$

Plugging back to Eq. 18, we obtain:

$$\nabla_{\boldsymbol{\theta}} J_{KL}(\boldsymbol{\theta}) = -\mathbb{E}_{t\sim\mathcal{U}[0,T], \boldsymbol{c}\sim p_c(\boldsymbol{c}), \boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})} \left[\omega(t) \frac{\sigma_t}{\alpha_t} \cdot \alpha_t \frac{\partial g(\boldsymbol{\theta}, \boldsymbol{c})}{\partial \boldsymbol{\theta}} \nabla \log p_t(\boldsymbol{x}_t | \boldsymbol{y}) \right] \tag{21}$$

$$= -\mathbb{E}_{t\sim\mathcal{U}[0,T], \boldsymbol{c}\sim p_c(\boldsymbol{c}), \boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})} \left[\omega(t) \frac{\partial g(\boldsymbol{\theta}, \boldsymbol{c})}{\partial \boldsymbol{\theta}} \sigma_t \nabla \log p_t(\boldsymbol{x}_t | \boldsymbol{y}) \right], \tag{22}$$

where $\boldsymbol{x}_t = \alpha_t g(\boldsymbol{\theta}, \boldsymbol{c}) + \sigma_t \boldsymbol{\epsilon}$. □

Below we reproduce two results, which state both SDS (Eq. 1) and VSD (Eq. 2) optimize for J_{KL}.

Lemma 2 (SDS minimizes J_{KL} [31])**.** *For any $\boldsymbol{\theta}$, we have $J_{SDS}(\boldsymbol{\theta}) = J_{KL}(\boldsymbol{\theta}) + const$.*

Proof. It is sufficient to show $\nabla_{\boldsymbol{\theta}} J_{SDS}(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} J_{KL}(\boldsymbol{\theta})$. By expansion:

$$\nabla_{\boldsymbol{\theta}} J_{SDS}(\boldsymbol{\theta}) = -\mathbb{E}_{t\sim\mathcal{U}[0,T], \boldsymbol{c}\sim p_c(\boldsymbol{c}), \boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})} \left[\omega(t) \frac{\partial g(\boldsymbol{\theta}, \boldsymbol{c})}{\partial \boldsymbol{\theta}} \left(\sigma_t \nabla \log p_t(\boldsymbol{x}_t | \boldsymbol{y}) - \boldsymbol{\epsilon} \right) \right] \tag{23}$$

$$= \underbrace{-\mathbb{E}_{t\sim\mathcal{U}[0,T], \boldsymbol{c}\sim p_c(\boldsymbol{c}), \boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})} \left[\omega(t) \frac{\partial g(\boldsymbol{\theta}, \boldsymbol{c})}{\partial \boldsymbol{\theta}} \sigma_t \nabla \log p_t(\boldsymbol{x}_t | \boldsymbol{y}) \right]}_{\nabla_{\boldsymbol{\theta}} J_{KL}(\boldsymbol{\theta})} + \underbrace{\mathbb{E}_{t\sim\mathcal{U}[0,T], \boldsymbol{c}\sim p_c(\boldsymbol{c}), \boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})} \left[\omega(t) \sigma_t \frac{\partial g(\boldsymbol{\theta}, \boldsymbol{c})}{\partial \boldsymbol{\theta}} \boldsymbol{\epsilon} \right]}_{=\mathbf{0}}, \tag{24}$$

where the second term equals $\mathbf{0}$ because $\boldsymbol{\epsilon}$ is zero mean and sampled independently. □

Lemma 3 (Single-particle VSD minimizes J_{KL} [56])**.** *For any $\boldsymbol{\theta}$, we have $J_{VSD}(\boldsymbol{\theta}) = J_{KL}(\boldsymbol{\theta}) + const$.*

Proof. It is sufficient to show $\nabla_{\boldsymbol{\theta}} J_{VSD}(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} J_{KL}(\boldsymbol{\theta})$. By a similar expansion:

$$\nabla_{\boldsymbol{\theta}} J_{VSD}(\boldsymbol{\theta}) = -\mathbb{E}_{t\sim\mathcal{U}[0,T], \boldsymbol{c}\sim p_c(\boldsymbol{c}), \boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})} \left[\omega(t) \frac{\partial g(\boldsymbol{\theta}, \boldsymbol{c})}{\partial \boldsymbol{\theta}} \left(\sigma_t \nabla \log p_t(\boldsymbol{x}_t | \boldsymbol{y}) - \sigma_t \nabla \log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t | \boldsymbol{c}, \boldsymbol{y}) \right) \right] \tag{25}$$

$$= \underbrace{-\mathbb{E}_{t\sim\mathcal{U}[0,T], \boldsymbol{c}\sim p_c(\boldsymbol{c}), \boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})} \left[\omega(t) \frac{\partial g(\boldsymbol{\theta}, \boldsymbol{c})}{\partial \boldsymbol{\theta}} \sigma_t \nabla \log p_t(\boldsymbol{x}_t | \boldsymbol{y}) \right]}_{\nabla_{\boldsymbol{\theta}} J_{KL}(\boldsymbol{\theta})} \tag{26}$$

$$+ \underbrace{\mathbb{E}_{t\sim\mathcal{U}[0,T], \boldsymbol{c}\sim p_c(\boldsymbol{c}), \boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})} \left[\omega(t) \frac{\partial g(\boldsymbol{\theta}, \boldsymbol{c})}{\partial \boldsymbol{\theta}} \sigma_t \nabla \log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t | \boldsymbol{c}, \boldsymbol{y}) \right]}_{=(a)} \tag{27}$$

Then we conclude the proof by showing $(a) = \mathbf{0}$ due to the fact that the first-order moment of score functions equals zero:

$$(a) = \mathbb{E}_{t\sim\mathcal{U}[0,T], \boldsymbol{c}\sim p_c(\boldsymbol{c})} \left[\omega(t) \frac{\sigma_t}{\alpha_t} \mathbb{E}_{\boldsymbol{x}_t \sim q_t^{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{c},\boldsymbol{y})} \left[\alpha_t \frac{\partial g(\boldsymbol{\theta}, \boldsymbol{c})}{\partial \boldsymbol{\theta}} \nabla \log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t | \boldsymbol{c}, \boldsymbol{y}) \right] \right] \tag{28}$$

$$= \mathbb{E}_{t\sim\mathcal{U}[0,T], \boldsymbol{c}\sim p_c(\boldsymbol{c})} \left[\omega(t) \frac{\sigma_t}{\alpha_t} \mathbb{E}_{\boldsymbol{x}_t \sim q_t^{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{c},\boldsymbol{y})} \left[\nabla_{\boldsymbol{\theta}} \log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t | \boldsymbol{c}, \boldsymbol{y}) \right] \right] \tag{29}$$

$$= \mathbb{E}_{t\sim\mathcal{U}[0,T], \boldsymbol{c}\sim p_c(\boldsymbol{c})} \left[\omega(t) \frac{\sigma_t}{\alpha_t} \int \frac{\nabla_{\boldsymbol{\theta}} q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t | \boldsymbol{c}, \boldsymbol{y})}{q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t | \boldsymbol{c}, \boldsymbol{y})} q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t | \boldsymbol{c}, \boldsymbol{y}) d\boldsymbol{x}_t \right] \tag{30}$$

$$= \mathbb{E}_{t\sim\mathcal{U}[0,T], \boldsymbol{c}\sim p_c(\boldsymbol{c})} \left[\omega(t) \frac{\sigma_t}{\alpha_t} \nabla_{\boldsymbol{\theta}} \int q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t | \boldsymbol{c}, \boldsymbol{y}) d\boldsymbol{x}_t \right] = \mathbf{0}, \tag{31}$$

where we use change of variables by reversing the chain rule in Eq. 29, and the last step is because the integral equals one, which is independent of $\boldsymbol{\theta}$. □

Remark 1. For multi-particle VSD, Lemma 3 may not hold. This is because the reverse chain rule in Eq. 29 is no longer applicable as $q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y})$ also becomes a function of $\boldsymbol{\theta}$.

Finally, we show that optimizing J_{KL} is equivalent to optimizing J_{MLE} (Eq. 5). First, recall that:

$$J_{MLE}(\boldsymbol{\theta}) = -\mathbb{E}_{t\sim\mathcal{U}[0,T],\boldsymbol{c}\sim p_c(\boldsymbol{c})}\left[\omega(t)\frac{\sigma_t}{\alpha_t}\mathbb{E}_{\boldsymbol{x}_t\sim q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y})}\left[\log p_t(\boldsymbol{x}_t|\boldsymbol{y})\right]\right]. \tag{32}$$

Then we state the following lemma:

Lemma 4 (J_{KL} is equivalent to maximal likelihood estimation)**.** *For any $\boldsymbol{\theta}$, we have $J_{MLE}(\boldsymbol{\theta}) = J_{KL}(\boldsymbol{\theta}) + const$.*

Proof. Again, we show $\nabla_{\boldsymbol{\theta}} J_{MLE}(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} J_{KL}(\boldsymbol{\theta})$:

$$\nabla_{\boldsymbol{\theta}} J_{MLE}(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} - \mathbb{E}_{t\sim\mathcal{U}[0,T],\boldsymbol{c}\sim p_c(\boldsymbol{c})}\left[\omega(t)\frac{\sigma_t}{\alpha_t}\mathbb{E}_{\boldsymbol{x}_t\sim q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y})}\left[\log p_t(\boldsymbol{x}_t|\boldsymbol{y})\right]\right] \tag{33}$$

$$= -\mathbb{E}_{t\sim\mathcal{U}[0,T],\boldsymbol{c}\sim p_c(\boldsymbol{c})}\left[\omega(t)\frac{\sigma_t}{\alpha_t}\mathbb{E}_{\boldsymbol{x}_t\sim q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y})}\left[\nabla_{\boldsymbol{\theta}}\log p_t(\boldsymbol{x}_t|\boldsymbol{y})\right]\right] \tag{34}$$

$$= -\mathbb{E}_{t\sim\mathcal{U}[0,T],\boldsymbol{c}\sim p_c(\boldsymbol{c})}\left[\omega(t)\frac{\sigma_t}{\alpha_t}\mathbb{E}_{\boldsymbol{x}_t\sim q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y})}\left[\alpha_t\frac{\partial g(\boldsymbol{\theta},\boldsymbol{c})}{\partial\boldsymbol{\theta}}\nabla\log p_t(\boldsymbol{x}_t|\boldsymbol{y})\right]\right] \tag{35}$$

$$= -\mathbb{E}_{t\sim\mathcal{U}[0,T],\boldsymbol{c}\sim p_c(\boldsymbol{c}),\boldsymbol{\epsilon}\sim\mathcal{N}(\boldsymbol{0},\boldsymbol{I})}\left[\omega(t)\frac{\partial g(\boldsymbol{\theta},\boldsymbol{c})}{\partial\boldsymbol{\theta}}\sigma_t\nabla\log p_t(\boldsymbol{x}_t|\boldsymbol{y})\right], \tag{36}$$

where the last step is basic reparameterization of $\boldsymbol{x}_t = \alpha_t\boldsymbol{x}_0 + \sigma_t\boldsymbol{\epsilon}$, and $\boldsymbol{x}_0 = g(\boldsymbol{\theta},\boldsymbol{c})$. □

As we argue in Sec. 3 (Eq. 5), the root reason J_{KL} degenerates to J_{MLE} is because the entropy term in J_{KL} becomes a constant independent of $\boldsymbol{\theta}$.

A.2. Derivation of Entropic Score Distillation

In this section, we derive the gradient for our entropy regularized objective (Eq. 8). We restate the entropy regularized objective (Eq. 7) below:

$$J_{Ent}(\boldsymbol{\theta},\lambda) = -\mathbb{E}_{t\sim\mathcal{U}[0,T],\boldsymbol{c}\sim p_c(\boldsymbol{c})}\left[\omega(t)\frac{\sigma_t}{\alpha_t}\mathbb{E}_{\boldsymbol{x}_t\sim q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y})}\log p_t(\boldsymbol{x}_t|\boldsymbol{y})\right] - \lambda\,\mathbb{E}_{t\sim\mathcal{U}[0,T]}\left[\omega(t)\frac{\sigma_t}{\alpha_t}H[q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})]\right], \tag{37}$$

where the entropy term $H[q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})]$ is defined as:

$$H\left[q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})\right] = -\mathbb{E}_{\boldsymbol{x}_t\sim q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})}\left[\log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})\right], \tag{38}$$

and distribution $q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})$ is defined as:

$$q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y}) = \int q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y})p_c(\boldsymbol{c})d\boldsymbol{c}. \tag{39}$$

Notice that $J_{Ent}(\boldsymbol{\theta},\lambda) = J_{MLE}(\boldsymbol{\theta}) - \lambda\,\mathbb{E}_{t\sim\mathcal{U}[0,T]}\left[\omega(t)\frac{\sigma_t}{\alpha_t}H[q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})]\right]$, therefore, to derive Eq. 8, we simply need the gradient of the entropy term:

Lemma 5 (Gradient of entropy)**.** *It holds that:*

$$\nabla_{\boldsymbol{\theta}} H\left[q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})\right] = -\mathbb{E}_{\boldsymbol{c}\sim p_c(\boldsymbol{c}),\boldsymbol{\epsilon}\sim\mathcal{N}(\boldsymbol{0},\boldsymbol{I})}\left[\alpha_t\frac{\partial g(\boldsymbol{\theta},\boldsymbol{c})}{\partial\boldsymbol{\theta}}\nabla\log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})\right]. \tag{40}$$

Proof. We expand entropy by reparameterization of $q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})$ as sampling two independent variables $\boldsymbol{c},\boldsymbol{\epsilon}$:

$$\nabla_{\boldsymbol{\theta}} H\left[q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})\right] = \nabla_{\boldsymbol{\theta}}\mathbb{E}_{\boldsymbol{c}\sim p_c(\boldsymbol{c}),\boldsymbol{\epsilon}\sim\mathcal{N}(\boldsymbol{0},\boldsymbol{I})}\left[-\log q_t^{\boldsymbol{\theta}}(\alpha_t g(\boldsymbol{\theta},\boldsymbol{c}) + \sigma_t\boldsymbol{\epsilon}|\boldsymbol{y})\right] \tag{41}$$

$$= -\mathbb{E}_{\boldsymbol{c}\sim p_c(\boldsymbol{c}),\boldsymbol{\epsilon}\sim\mathcal{N}(\boldsymbol{0},\boldsymbol{I})}\left[\nabla_{\boldsymbol{\theta}}\log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y}) + \alpha_t\frac{\partial g(\boldsymbol{\theta},\boldsymbol{c})}{\partial\boldsymbol{\theta}}\nabla_{\boldsymbol{x}_t}\log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})\right]\Bigg|_{\boldsymbol{x}_t=\alpha_t g(\boldsymbol{\theta},\boldsymbol{c})+\sigma_t\boldsymbol{\epsilon}} \tag{42}$$

$$= -\underbrace{\mathbb{E}_{\boldsymbol{x}_t\sim q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})}\left[\nabla_{\boldsymbol{\theta}}\log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})\right]}_{=(a)} - \mathbb{E}_{\boldsymbol{c}\sim p_c(\boldsymbol{c}),\boldsymbol{\epsilon}\sim\mathcal{N}(\boldsymbol{0},\boldsymbol{I})}\left[\alpha_t\frac{\partial g(\boldsymbol{\theta},\boldsymbol{c})}{\partial\boldsymbol{\theta}}\nabla_{\boldsymbol{x}_t}\log q_t^{\boldsymbol{\theta}}(\alpha_t g(\boldsymbol{\theta},\boldsymbol{c}) + \sigma_t\boldsymbol{\epsilon}|\boldsymbol{\theta})\right], \tag{43}$$

where it is noteworthy that $\nabla_{\boldsymbol{x}_t} \log q_t^{\boldsymbol{\theta}}$ simply denotes the score function of $q_t^{\boldsymbol{\theta}}$ by explicitly indicating the derivative is taken in terms of $\boldsymbol{x}_t$. Eq. 42 is obtained by path derivative. It remains to show $(a) = \mathbf{0}$. We recall that the first-order moment of a score function equals to zero:

$$(a) = \int \nabla_{\boldsymbol{\theta}} \log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y}) q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y}) d\boldsymbol{x}_t = \int \frac{\nabla_{\boldsymbol{\theta}} q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})}{q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})} q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y}) d\boldsymbol{x}_t \tag{44}$$

$$= \nabla_{\boldsymbol{\theta}} \int q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y}) d\boldsymbol{x}_t \tag{45}$$

$$= \mathbf{0}, \tag{46}$$

where the last step involves a change of variable and the integral turns out to be independent of $\boldsymbol{\theta}$. □

As a consequence, we can conclude the update rule yielded by Eq. 7 in the following theorem:

Theorem 2 (Entropic Score Distillation)**.** *For any $\boldsymbol{\theta}$ and $\lambda \in \mathbb{R}$, the following holds:*

$$\nabla_{\boldsymbol{\theta}} J_{Ent}(\boldsymbol{\theta}, \lambda) = -\mathbb{E}_{t\sim\mathcal{U}[0,T], \boldsymbol{c}\sim p_c(\boldsymbol{c}), \boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})} \left[\omega(t) \frac{\partial g(\boldsymbol{\theta}, \boldsymbol{c})}{\partial \boldsymbol{\theta}} \left(\sigma_t \nabla \log p_t(\boldsymbol{x}_t|\boldsymbol{y}) - \lambda \sigma_t \nabla \log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y}) \right) \right], \tag{47}$$

where $\boldsymbol{x}_t = \alpha_t \boldsymbol{x}_0 + \sigma_t \boldsymbol{\epsilon}$, and $\boldsymbol{x}_0 = g(\boldsymbol{\theta}, \boldsymbol{c})$.

Proof. Since $J_{Ent}(\boldsymbol{\theta}, \lambda) = J_{MLE}(\boldsymbol{\theta}) - \lambda \mathbb{E}_{t\sim\mathcal{U}[0,T]} \left[\omega(t) \frac{\sigma_t}{\alpha_t} H[q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})] \right]$, by Lemma 4 and Lemma 5:

$$\nabla_{\boldsymbol{\theta}} J_{Ent}(\boldsymbol{\theta}, \lambda) = \nabla_{\boldsymbol{\theta}} J_{MLE}(\boldsymbol{\theta}) - \lambda \mathbb{E}_{t\sim\mathcal{U}[0,T]} \left[\omega(t) \frac{\sigma_t}{\alpha_t} \nabla_{\boldsymbol{\theta}} H[q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})] \right] \tag{48}$$

$$= -\mathbb{E}_{t\sim\mathcal{U}[0,T], \boldsymbol{c}\sim p_c(\boldsymbol{c}), \boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})} \left[\omega(t) \frac{\partial g(\boldsymbol{\theta}, \boldsymbol{c})}{\partial \boldsymbol{\theta}} \sigma_t \nabla \log p_t(\boldsymbol{x}_t|\boldsymbol{y}) \right] \tag{49}$$

$$+ \mathbb{E}_{t\sim\mathcal{U}[0,T], \boldsymbol{c}\sim p_c(\boldsymbol{c}), \boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})} \left[\omega(t) \frac{\partial g(\boldsymbol{\theta}, \boldsymbol{c})}{\partial \boldsymbol{\theta}} \lambda \sigma_t \nabla \log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y}) \right], \tag{50}$$

by which we conclude the proof by merging two expectations. □

A.3. Justification of Classifier-Free Guidance Trick

In this section, we first prove Theorem 1 and show that CFG trick (Eq. 9) can be utilized to implement ESD. To begin with, we define another type of KL divergence as below:

$$\overline{J_{KL}} = \mathbb{E}_{t\sim\mathcal{U}[0,T]} \left[\omega(t) \frac{\sigma_t}{\alpha_t} \mathcal{D}_{\mathrm{KL}}(q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y}) \| p_t(\boldsymbol{x}_t|\boldsymbol{y})) \right] \tag{51}$$

Then we present the following lemma, which represents the gradient of $\overline{J_{KL}}$:

Lemma 6 (Gradient of $\overline{J_{KL}}$)**.** *It holds that:*

$$\nabla_{\boldsymbol{\theta}} \overline{J_{KL}} = -\mathbb{E}_{t\sim\mathcal{U}[0,T], \boldsymbol{c}\sim p_c(\boldsymbol{c}), \boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})} \left[\omega(t) \frac{\partial g(\boldsymbol{\theta}, \boldsymbol{c})}{\partial \boldsymbol{\theta}} \left(\sigma_t \nabla \log p_t(\boldsymbol{x}_t|\boldsymbol{y}) - \sigma_t \nabla \log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y}) \right) \right], \tag{52}$$

where $\boldsymbol{x}_t = \alpha_t \boldsymbol{x}_0 + \sigma_t \boldsymbol{\epsilon}$, and $\boldsymbol{x}_0 = g(\boldsymbol{\theta}, \boldsymbol{c})$.

Proof. We prove by showing $\overline{J_{KL}}$ is a special case of J_{Ent} when setting $\lambda = 1$:

$$\overline{J_{KL}}(\boldsymbol{\theta}) = \mathbb{E}_{t\sim\mathcal{U}[0,T], \boldsymbol{c}\sim p_c(\boldsymbol{c}), \boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})} \left[\omega(t) \frac{\sigma_t}{\alpha_t} \log \frac{q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})}{p_t(\boldsymbol{x}_t|\boldsymbol{y}))} \right] \tag{53}$$

$$= -\mathbb{E}_{t\sim\mathcal{U}[0,T], \boldsymbol{c}\sim p_c(\boldsymbol{c}), \boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})} \left[\omega(t) \frac{\sigma_t}{\alpha_t} \log p_t(\boldsymbol{x}_t|\boldsymbol{y})) \right] + \mathbb{E}_{t\sim\mathcal{U}[0,T], \boldsymbol{c}\sim p_c(\boldsymbol{c}), \boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})} \left[\omega(t) \frac{\sigma_t}{\alpha_t} \log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y}) \right] \tag{54}$$

$$= J_{MLE}(\boldsymbol{\theta}) + \mathbb{E}_{t\sim\mathcal{U}[0,T]} \mathbb{E}_{\boldsymbol{x}_t\sim q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})} \left[\omega(t) \frac{\sigma_t}{\alpha_t} \log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y}) \right] \tag{55}$$

$$= J_{MLE}(\boldsymbol{\theta}) - \mathbb{E}_{t\sim\mathcal{U}[0,T]} \left[\omega(t) \frac{\sigma_t}{\alpha_t} H\left[q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y}) \right] \right] = J_{Ent}(\boldsymbol{\theta}, 1) \tag{56}$$

□

Now we prove Theorem 1 using previous results:

Proof of Theorem 1. It is sufficient to show that $\nabla_{\boldsymbol{\theta}} J_{Ent}(\boldsymbol{\theta}, \lambda) = \lambda \nabla_{\boldsymbol{\theta}} \overline{J_{KL}}(\boldsymbol{\theta}) + (1-\lambda)\nabla_{\boldsymbol{\theta}} J_{KL}(\boldsymbol{\theta})$. By Lemma 1, 6, as well as Theorem 2, we can obtain:

$$\lambda \nabla_{\boldsymbol{\theta}} \overline{J_{KL}}(\boldsymbol{\theta}) + (1-\lambda)\nabla_{\boldsymbol{\theta}} J_{KL}(\boldsymbol{\theta}) = -\lambda\, \mathbb{E}_{t\sim\mathcal{U}[0,T], \boldsymbol{c}\sim p_c(\boldsymbol{c}), \boldsymbol{\epsilon}\sim\mathcal{N}(\boldsymbol{0},\boldsymbol{I})} \left[\omega(t)\frac{\partial g(\boldsymbol{\theta},\boldsymbol{c})}{\partial \boldsymbol{\theta}} \left(\sigma_t \nabla \log p_t(\boldsymbol{x}_t|\boldsymbol{y}) - \sigma_t \nabla \log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})\right)\right] \tag{57}$$

$$- (1-\lambda)\, \mathbb{E}_{t\sim\mathcal{U}[0,T], \boldsymbol{c}\sim p_c(\boldsymbol{c}), \boldsymbol{\epsilon}\sim\mathcal{N}(\boldsymbol{0},\boldsymbol{I})} \left[\omega(t)\frac{\partial g(\boldsymbol{\theta},\boldsymbol{c})}{\partial \boldsymbol{\theta}} \sigma_t \nabla \log p_t(\boldsymbol{x}_t|\boldsymbol{y})\right] \tag{58}$$

$$= \nabla_{\boldsymbol{\theta}} J_{Ent}(\boldsymbol{\theta}, \lambda), \tag{59}$$

by merging two expectations. □

Further on, our CFG trick implementation of ESD (Eq. 9) can be regarded as a corollary of Theorem 1 and Lemma 3:

Theorem 3 (Classifier-Free Guidance Trick)**.** *For any $\boldsymbol{\theta}$ and $\lambda \in \mathbb{R}$, $\nabla_{\boldsymbol{\theta}} J_{Ent}(\boldsymbol{\theta}, \lambda)$ equals to the following:*

$$-\mathbb{E}_{t\sim\mathcal{U}[0,T], \boldsymbol{c}\sim p_c(\boldsymbol{c}), \boldsymbol{\epsilon}\sim\mathcal{N}(\boldsymbol{0},\boldsymbol{I})} \left[\omega(t)\frac{\partial g(\boldsymbol{\theta},\boldsymbol{c})}{\partial \boldsymbol{\theta}} \left(\sigma_t \nabla \log p_t(\boldsymbol{x}_t|\boldsymbol{y}) - \lambda\sigma_t \nabla \log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y}) - (1-\lambda)\sigma_t \nabla \log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y})\right)\right], \tag{60}$$

where $\boldsymbol{x}_t = \alpha_t \boldsymbol{x}_0 + \sigma_t \boldsymbol{\epsilon}$, and $\boldsymbol{x}_0 = g(\boldsymbol{\theta}, \boldsymbol{c})$.

Proof. By Theorem 1, we know that $\nabla_{\boldsymbol{\theta}} J_{Ent}(\boldsymbol{\theta}, \lambda) = \lambda \nabla_{\boldsymbol{\theta}} \overline{J_{KL}}(\boldsymbol{\theta}) + (1-\lambda)\nabla_{\boldsymbol{\theta}} J_{KL}(\boldsymbol{\theta})$. Moreover, by Lemma 3, we have $\nabla_{\boldsymbol{\theta}} J_{KL}(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} J_{VSD}(\boldsymbol{\theta})$. As a result, the following can be derived:

$$\nabla_{\boldsymbol{\theta}} J_{Ent}(\boldsymbol{\theta}, \lambda) = \nabla_{\boldsymbol{\theta}} \overline{J_{KL}}(\boldsymbol{\theta}) + (1-\lambda)\nabla_{\boldsymbol{\theta}} J_{VSD}(\boldsymbol{\theta}) \tag{61}$$

$$= -\lambda\, \mathbb{E}_{t\sim\mathcal{U}[0,T], \boldsymbol{c}\sim p_c(\boldsymbol{c}), \boldsymbol{\epsilon}\sim\mathcal{N}(\boldsymbol{0},\boldsymbol{I})} \left[\omega(t)\frac{\partial g(\boldsymbol{\theta},\boldsymbol{c})}{\partial \boldsymbol{\theta}} \left(\sigma_t \nabla \log p_t(\boldsymbol{x}_t|\boldsymbol{y}) - \sigma_t \nabla \log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{y})\right)\right] \tag{62}$$

$$- (1-\lambda)\, \mathbb{E}_{t\sim\mathcal{U}[0,T], \boldsymbol{c}\sim p_c(\boldsymbol{c}), \boldsymbol{\epsilon}\sim\mathcal{N}(\boldsymbol{0},\boldsymbol{I})} \left[\omega(t)\frac{\partial g(\boldsymbol{\theta},\boldsymbol{c})}{\partial \boldsymbol{\theta}} \left(\sigma_t \nabla \log p_t(\boldsymbol{x}_t|\boldsymbol{y}) - \sigma_t \nabla \log q_t^{\boldsymbol{\theta}}(\boldsymbol{x}_t|\boldsymbol{c},\boldsymbol{y})\right)\right], \tag{63}$$

as desired after merging two expectations. □

B. Illustrative Examples

Gaussian Distribution Fitting. In this section, we provide the necessary details on Fig. 3, where we fit a 2D Gaussian distribution via SDS, VSD, and ESD. Suppose the targeted Gaussian distribution is $p_0(\boldsymbol{x}_0) = \mathcal{N}(\boldsymbol{x}_0|\boldsymbol{\mu}^*, \boldsymbol{\Sigma}^*)$, where $\boldsymbol{\mu}^* \in \mathbb{R}^D$ is the mean vector and $\boldsymbol{\Sigma}^* \in \mathbb{R}^{D\times D}$ is the positive definite covariance matrix. Define differentiable function $g(\{\boldsymbol{b}, \boldsymbol{A}\}, \boldsymbol{c}) = \boldsymbol{b} + \boldsymbol{A}\boldsymbol{c}$, where $\boldsymbol{b}$ and $\boldsymbol{A}$ are parameters to be fitted, and $\boldsymbol{c} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ is a random variable sampled from a standard Gaussian distribution. It is obvious that probability density function of $g(\{\boldsymbol{b}, \boldsymbol{A}\}, \boldsymbol{c})$ is $q_0^{\boldsymbol{b},\boldsymbol{A}}(\boldsymbol{x}_0) = \mathcal{N}(\boldsymbol{x}_0|\boldsymbol{b}, \boldsymbol{A}\boldsymbol{A}^\top)$. Our objective is to match p_0 and $q_0^{\boldsymbol{b},\boldsymbol{A}}$ by optimizing parameters $\boldsymbol{b}$ and $\boldsymbol{A}$ with SDS, VSD, and ESD. Notice that diffusion perturbed p_0 and $q_0^{\boldsymbol{b},\boldsymbol{A}}$ are still Gaussian distributions. And the score functions used in score distillation can be computed in the closed form:

$$\nabla \log p_t(\boldsymbol{x}_t) = (\alpha_t^2 \boldsymbol{\Sigma}^* + \sigma_t^2 \boldsymbol{I})^{-1}(\alpha_t \boldsymbol{\mu}^* - \boldsymbol{x}_t), \tag{64}$$

$$\nabla \log q_t^{\boldsymbol{b},\boldsymbol{A}}(\boldsymbol{x}_t) = (\alpha_t^2 \boldsymbol{A}\boldsymbol{A}^\top + \sigma_t^2 \boldsymbol{I})^{-1}(\alpha_t \boldsymbol{b} - \boldsymbol{x}_t), \quad \nabla \log q_t^{\boldsymbol{b},\boldsymbol{A}}(\boldsymbol{x}_t|\boldsymbol{c}) = \frac{\alpha_t(\boldsymbol{b} + \boldsymbol{A}\boldsymbol{c}) - \boldsymbol{x}_t}{\sigma_t^2}. \tag{65}$$

In our experiments, we run score distillation for 2k steps with 100 warm-up steps. The learning rate is set to 0.01. We observe that SDS and VSD have similar convergence behavior as maximal likelihood methods. When increasing λ from 0 to 1, i.e., enhancing the effect of entropy maximization, the fitted distribution gradually covers the targeted support.

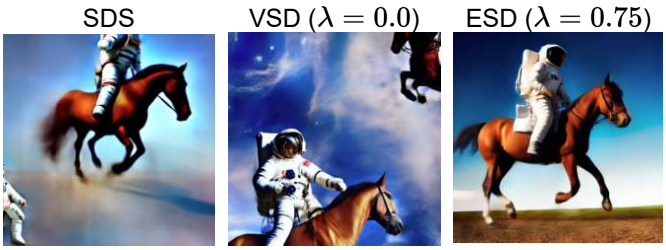

Figure 8. **Image Reconstruction Example.** We leverage SDS, VSD, and ESD to recover a high-resolution 2D image by matching the distribution of its random crops with a pre-trained text-conditioned diffusion model. Prompt: An astronaut riding a horse in space.

2D Image Reconstruction. We also conduct 2D image experiments to demonstrate the working mechanism of ESD. We focus on the image reconstruction task via partial observations [6, 57], where the optimized parameters represent a high-resolution image and a random small window of the image will be rendered at each iteration of score distillation. Formally, let $\boldsymbol{\theta}$ denote the high-resolution image, and $g(\boldsymbol{\theta}, \boldsymbol{m}) = \boldsymbol{\theta} \odot \boldsymbol{m}$, where $\boldsymbol{m}$ is a random binary mask. We choose $\nabla \log p_t(\boldsymbol{x}_t|\boldsymbol{y})$ as a pre-trained text-to-image diffusion model, and $\boldsymbol{y}$ is the text prompt, specified as "An astronaut riding a horse in space" in our experiments. Akin to [56], $\nabla \log q_t(\boldsymbol{x}_t)$ and $\nabla \log q_t(\boldsymbol{x}_t|\boldsymbol{m})$ are fitted via LoRA using cropped images. During training, we fix training steps as 10k, learning rate as 1e-2 for $\boldsymbol{\theta}$, and 1e-4 for LoRA. We also apply the cosine learning rate scheduler with 100 warm-up steps. We present qualitative results in Fig. 8. It demonstrates that SDS and VSD cause "Janus"-like problems where each image contains duplicated instances while ESD avoids such issues and generates only one target object.

C. Experiment Details

In this section, we provide more details on the implementation of ESD and the compared baseline methods. All of them are implemented under the `threestudio` framework and include three stages: coarse generation, geometry refinement, and texture refinement, following [56]. For the coarse generation stage, we adopt foreground-background disentangled hash-encoded NeRF [28] as the underlying 3D representation, and DMTet [40] for two refinement stages. All scenes are trained for 25k steps for the coarse stage, 10k steps for geometry refinement, and 30k steps for texture refinement, for the sake of fair comparison. At each iteration, we randomly render one view (i.e., batch size equals one). We progressively adjust rendering resolution: within the first 5k steps, we render at 64×64 resolution and increase to 256×256 resolution afterward.

SDS [31]. Following the original paper, we set the CFG weight to 100. Additionally, we encourage sparsity of the density field and penalize the mismatch between orientation and predicted normal maps. Lighting augmentation is also enabled for SDS. The geometry refinement step is directly borrowed from VSD: a DMTet is initialized by NeRF's density field via marching cube while the end-to-end optimization with SDS is then conducted on the geometry representation for both geometry and texture.

VSD [56]. We reuse the standard setting of VSD for all three stages. In particular, we fix the CFG coefficient to 7.5 and only use single-particle VSD, conforming with our theoretical analysis. During the geometry refinement stage, we adopt SDS guidance instead of VSD.

Debiased-SDS [12]. Our implementation of Debiased-SDS is built upon SDS. We enable both score debiasing and prompt debiasing. For score debiasing, we follow the default setting and linearly increase the absolute threshold for gradient clipping from 0.5 to 2.0. All other hyperparameters follow from SDS.

Perp-Neg [2]. Perp-Neg implementation is based on SDS as well. As suggested by the original paper, in positive prompts, we leverage weights $r_{interp} = 1 - 2|\text{azimuth}|/\pi$ for front-side prompt interpolation and $r_{interp} = 2 - 2|\text{azimuth}|/\pi$ for side-back interpolation. In negative prompts, the interpolating function is chosen as the shifted exponential function $\alpha \exp(-\beta r_{interp}) + \gamma$. Specifically, we choose $\alpha_{sf} = 1, \beta_{sf} = 0.5, \gamma_{sf} = -0.606, \alpha_{fsb} = 1, \beta_{fsb} = 0.5, \gamma_{fsb} = 0.967, \alpha_{fs} = 4, \beta_{fs} = 0.5, \gamma_{fs} = -2.426, \alpha_{sf} = 4, \beta_{sf} = 0.5, \gamma_{sf} = -2.426$. See [2] for more details on the meaning of these hyperparameters.

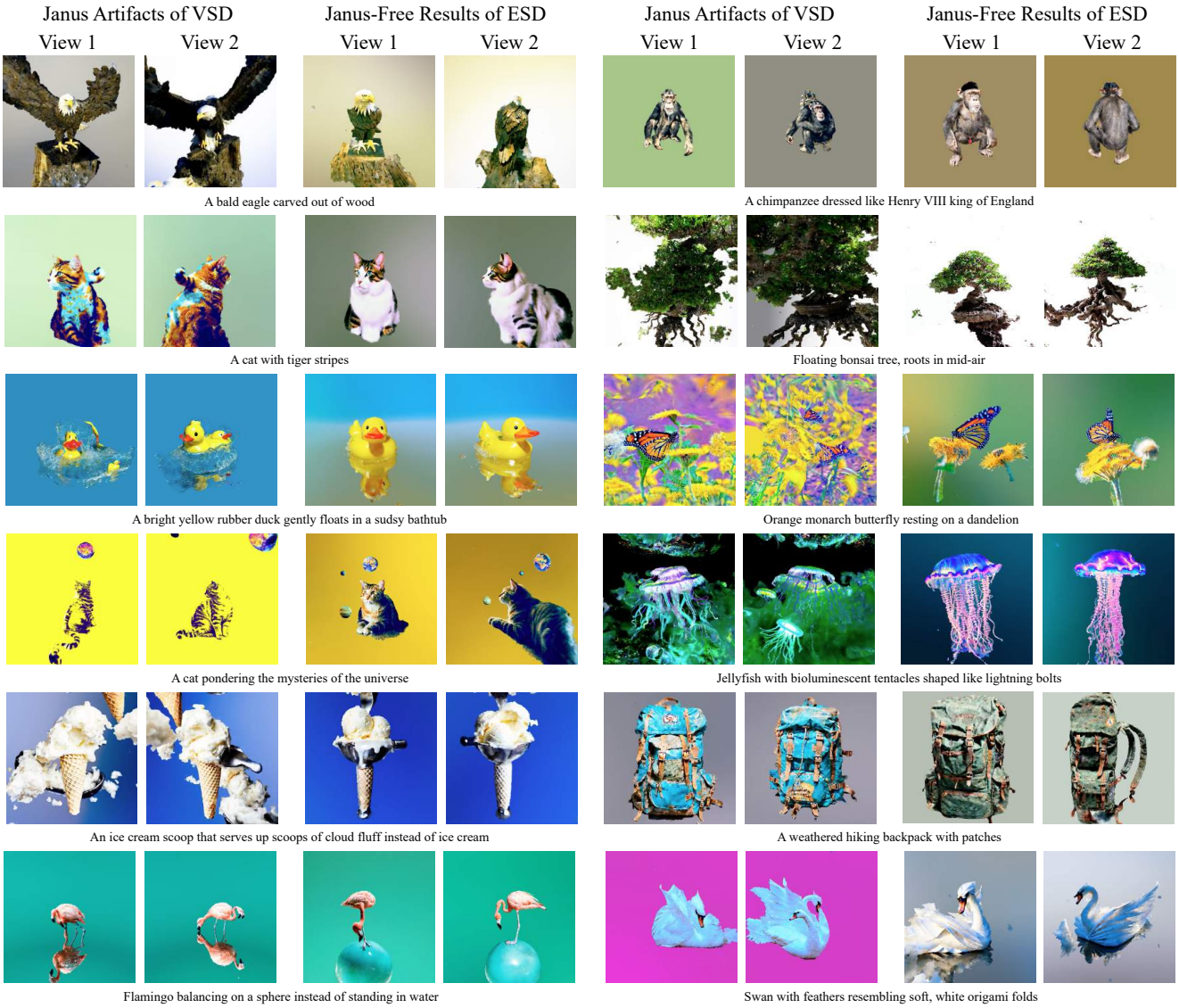

Figure 9. **More Qualitative Results.** We present the two views of each object synthesized by VSD (ProlificDreamer) and our method, respectively. Best view in an electronic copy.

ESD. Our ESD implementation is similar to VSD. We leverage the extrinsics matrix (4×4) as the camera pose embedding, and condition the diffusion model by replacing its class embedding branch. We introduce CFG trick to linearly mix camera-conditioned and unconditioned score functions fine-tuned with rendered images. We find CFG 0.5 generally yields desirable results. We also set the probability of unconditioned training to 0.5. In particular, view-dependent prompting is disabled for the fine-tuned score function.

D. More Qualitative Results

In this section, we present more qualitative results in Fig. 9. All text prompts are generated by GPT-4V and directly picked from [58]. We mainly compare ESD with VSD to highlight the influence of the entropy regularization term. The observation is consistent with our main text. The outcomes of VSD often exhibit broken geometries, duplicated objects, and multiple signature views, which contradict the inherent characteristics of the generated subjects. ESD can effectively mitigate "Janus" issues and generate more realistic contents.

E. Numerical Evaluation

Human Evaluation Criteria. In our human evaluation of Successful generation Rate (SR), a text prompt is labeled as "successfully generated" if at least one of three random seeds yields a generation satisfying the criteria: ([2], Appendix A.2):

1. The rendered images show the requested object(s), which is positioned on the correct view.
2. The rendered images do not show hallucination including counterfactual details, for example, a panda has three ears.
3. The rendered images do not have unrealistic color or texture or massive floaters.

Extension of Tab. 1. In Tab. 2, we include standard deviations of all numerical results presented in Tab. 1. We note that ESD exhibits a smaller variance of different metrics, indicating its training might be well-regularized and more robust.

Table 2. **Quantitative Comparisons.** (↓) means the lower the better, and (↑) means the higher the better.

	CLIP (↓)	FID (↓)	IQ (↓)	IV (↑)	IG (↑)	SR (↑)
SDS [31]	0.737±0.068	291.860±61.242	4.295±0.419	**4.8552±0.342**	0.123±0.053	15.00%
VSD [56]	0.725±0.072	265.141±58.549	3.149±1.234	3.5712±1.345	0.137±0.061	19.17%
ESD (Ours)	**0.714±0.065**	**235.915±56.558**	**3.135±1.088**	4.0314±1.285	**0.327±0.185**	**55.83%**

Breakdown Table. We provide a breakdown table to present quantitative evaluations of results in Fig. 4. The numbers are reported in Tab. 3. The conclusion is consistent with our argument in Sec. 7. Our ESD consistently outperforms all the compared baseline methods, especially in FID and IG. This implies that ESD effectively boosts the view diversity and accurately matches the distribution between pre-trained image distribution and rendered image distribution.

Table 3. **Quantitative Comparisons.** (↓) means the lower the better, and (↑) means the higher the better.

	CLIP (↓)	FID (↓)	IQ (↓)	IV (↑)	IG (↑)	CLIP (↓)	FID (↓)	IQ (↓)	IV (↑)	IG (↑)
	Michelangelo style statue of dog reading news on a cellphone					A rabbit, animated movie character, high detail 3d model				
SDS [31]	0.694	365.304	4.469	**5.119**	0.145	**0.712**	200.084	4.365	**4.970**	**0.138**
VSD [56]	0.758	296.168	**2.514**	3.041	0.209	0.720	150.120	**1.083**	1.173	0.083
Debiased-SDS [12]	0.778	351.493	4.058	4.814	0.186	0.735	216.058	4.443	4.857	0.093
Perp-Neg [2]	0.793	306.918	3.970	4.572	0.151	0.727	176.279	2.453	2.665	0.086
ESD (Ours)	**0.685**	**292.716**	2.523	4.080	**0.617**	0.725	**149.763**	1.385	1.567	0.132
	A rotary telephone carved out of wood					A plush dragon toy				
SDS [31]	0.853	309.929	3.478	4.179	0.202	0.889	243.984	4.622	**5.008**	0.084
VSD [56]	0.855	305.920	3.469	4.214	0.214	0.821	273.495	**4.382**	4.728	0.078
Debiased-SDS [12]	0.927	313.893	4.098	4.201	0.025	0.878	262.474	4.827	4.954	0.026
Perp-Neg [2]	0.868	308.554	3.488	4.021	0.153	0.839	309.276	4.691	4.816	0.027
ESD (Ours)	**0.846**	**299.578**	**3.332**	**4.439**	**0.366**	**0.815**	**237.518**	4.436	4.971	**0.121**

F. Limitations and Failure Cases

We note that by Theorem 1, ESD still optimizes for a mode-seeking KL divergence. This suggests that ESD may still lead to mode collapse especially when the target image distribution is overly concentrated on one peak [36]. Careful tuning of λ is also necessary to balance the per-view sharpness/details and cross-view diversity. It also remains open whether ESD can further benefit multi-particle VSD or amortized text-to-3D training [23].

Below we present a failure case produced by our ESD in Fig. 10, where the back view of the marble still contains a mouse face while the side views exhibit duplicate ears. We point out that even though ESD can encourage diversity among views, however, it may still incline to one mode when the target image distribution is overwhelmingly concentrated at one point. The specified text prompt in Fig. 10 is in this case as we observe the majority of sampled images from a pre-trained diffusion model with the corresponding prompt are the frontal views of a marble mouse.

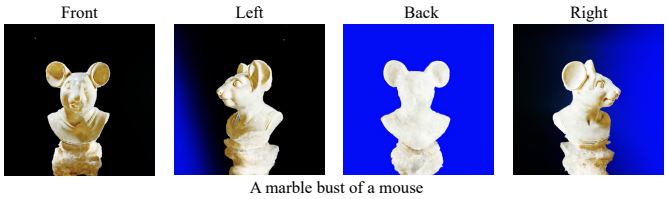

Figure 10. **Failure case.** We present four views of a failure case yielded by ESD with prompt "A marble bust of a mouse" and CFG weight $\lambda = 0.5$.